xplaineff: Regional Feature Effects for Better Model Explanations

by Zizheng Zhang

Abstract

Model-agnostic feature effect methods help analysts inspect how a fitted machine learning model's predictions depend on its inputs. Global feature effect methods such as partial dependence (PD) and accumulated local effects (ALE) summarize model behavior for prespecified features, but they average over potentially heterogeneous subgroups and can therefore mask feature interactions. The package **xplaineff** implements GADGET (Generalized Additive Decomposition of Global EffecTs), which recursively partitions the feature space to obtain regions in which feature effects are more homogeneous across observations and therefore easier to interpret. **xplaineff**'s core contribution is a fast, extensible R implementation for constructing regional PD/ALE explanations and visualizing how feature effects change across the resulting hierarchy of subregions. This paper summarizes the GADGET methodology, describes the design of the **xplaineff** package, and evaluates it through runtime benchmarks, a categorical level-ordering diagnostic, and a worked example. In the benchmarked settings, **xplaineff** is competitive with established R workflows for global PD/ALE computation. Its regional split-search module is faster than the Python package **effector**. The categorical diagnostic shows that data-driven `mds` and `pca` level orderings improve ALE split recovery when the remaining features carry information about the unordered levels.

Keywords: interpretable machine learning, feature effects, ALE, partial dependence, interaction detection, R.

1 Introduction

Modern predictive models are often used not only for prediction, but also for model inspection: analysts need tools that reveal how fitted predictions relate to input features (Molnar, 2019). Model-agnostic feature effect methods address this need by summarizing how a prediction changes when one feature is varied. As a running example, we use the hourly Capital Bikeshare data of Fanaee-T and Gama (2014), available as **ISLR2**'s `Bikeshare` data (James et al., 2022), where the task is to predict the number of bike rentals in a given hour from calendar and weather variables; Appendix I describes the features in detail. Here an analyst may ask how predicted rentals change with hour or temperature, and whether these effects differ between working days and non-working days. A single global hour curve is easy to communicate, but it can average over exactly this kind of working-day difference.

Two widely used feature effect methods are **partial dependence (PD)** (Friedman, 2001; Hastie et al., 2009) and **accumulated local effects (ALE)** (Apley and Zhu, 2020). PD plots show the average prediction when one feature is varied and the remaining features are marginalized. ALE plots accumulate local prediction differences along the feature axis and are often preferred when features are correlated, because they do not evaluate the model in low-density regions of the feature space and thereby avoid extrapolation (Apley and Zhu, 2020). Both methods, however, aggregate potentially heterogeneous local effects into a single feature-level summary and can therefore obscure region-specific patterns.

Individual conditional expectation (ICE) curves make such heterogeneity visible for PD by showing observation-level effect curves before averaging (Goldstein et al., 2015); Figure 2 shows these ICE curves for the bike-sharing example. Robust and Heterogeneity-aware Accumulated Local Effects (RHALE) provides an analogous diagnostic for ALE by estimating within-bin variability of observation-level local effects, so that even a smooth or near-zero accumulated curve can reveal heterogeneous local changes (Gkolemis et al., 2023a). Compared with global PD/ALE summaries, ICE and RHALE provide more nuanced

local information and can hint at interactions, for example through different ICE curve shapes or large within-bin variability. However, they do not identify which other feature the feature of interest interacts with, and they become hard to read once all observations are shown. Pairwise PD or ALE displays can visualize selected two-feature interactions directly, but the interaction pair must be chosen in advance, and enumerating many two-dimensional displays scales poorly when several candidate interaction features are plausible. We therefore target a regional approach between a single global curve and many local observation-level explanations: a user-controlled hierarchy of subregions in which feature effects are more homogeneous.

The GADGET (Generalized Additive Decomposition of Global EffecTs) algorithm (Herbinger et al., 2024) finds regions by recursively partitioning the input space such that feature effects are more homogeneous. GADGET starts from the full data and, within each resulting region, evaluates candidate splits over the user-specified split features. At each step, it selects the split that most reduces the heterogeneity of the effects of the features of interest. In the bike-sharing example shown later in Figures 1 and 2, the root split separates cooler and warmer observations, and the second-level splits refine these regions by season and working-day status before the regional hour effects are plotted. The result is a tree whose internal nodes describe where region-specific effects differ, and whose leaves provide regional effect curves that track local model behavior more closely than a single global average.

Contributions. The **xplaineff** package implements GADGET-style regional feature effects in R with the following contributions:

- **Unified R interface for regional PD and ALE:** the package provides a common API for constructing, inspecting, and plotting regional PD and ALE explanations, with separate user controls for the features whose effects are explained and the features allowed to define regions. Internally, a strategy-based design separates tree-growing logic from effect-specific computations, so the same interface supports method-specific PD and ALE calculations and future extensions.
- **Efficient split search:** for both PD and ALE, candidate splits are evaluated in C++ by updating the counts, sums, and sums of squares needed to compute child-node variances, rather than recomputing heterogeneity from scratch. For PD, these updates are applied to mean-centered ICE values; for ALE, they are applied to precomputed local effects within intervals, with the ALE self-feature correction derived in Appendix II.
- **Categorical split handling:** because ALE for categorical features requires an ordering of levels and no single ordering is generally canonical (Apley and Zhu, 2020), **xplaineff** makes this choice explicit. The package offers four strategies: keeping the existing level order, random permutation, and two data-driven orderings that compute pairwise distances between levels from the conditional distributions of the remaining features and embed the resulting distance matrix into one dimension via multidimensional scaling (MDS) or principal component analysis (PCA).
- **Empirical evaluation:** the paper separates runtime evaluation into global PD/ALE computation against established R packages and regional split search against the Python package **effector**. It further compares the categorical feature level ordering strategies for ALE implemented in **xplaineff** across raw, random, MDS, and PCA-based orderings, evaluating how each strategy recovers a known factor level grouping in simulation.

The paper is structured as follows. Section 2 introduces PD, ICE, ALE, and the GADGET framework for regional effect decomposition. Section 3 reviews and compares existing feature effect software in R and Python. Section 4 describes the package design, main API, and the bike-sharing running example. Section 5 reports empirical benchmarks against established R and Python baselines. Section 6 concludes.

2 Methodology

Notation. Let $\mathcal{X} := \mathcal{X}_1 \times \mathcal{X}_2 \times \cdots \times \mathcal{X}_p \subseteq \mathbb{R}^p$ be the feature space and $\mathcal{Y}$ the target space (e.g., $\mathcal{Y} \subseteq \mathbb{R}$ for regression tasks). We denote the random feature vector by $X = (X_1, \ldots, X_p)^\top$ and the target by the random variable Y, with joint distribution $P_{X,Y}$ on $\mathcal{X} \times \mathcal{Y}$. The observed data $D = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$ are i.i.d. realizations of (X, Y) drawn from $P_{X,Y}$, where $\mathbf{x}^{(i)} = (x_1^{(i)}, \ldots, x_p^{(i)})^\top$ is the i-th observation. For an arbitrary feature index set $W \subseteq \{1, \ldots, p\}$, $\mathbf{x}_W$ and X_W refer to the features in W. A leading minus denotes the complement, so $\mathbf{x}_{-W}$ and X_{-W} refer to all features outside W. For a single feature j, we write x_j, X_j, and $\mathbf{x}_{-j}$, X_{-j}, respectively. Throughout the paper, we consider an already trained supervised machine learning model with prediction function denoted by $\hat{f} : \mathcal{X} \to \mathcal{Y}$. We write $S \subseteq \{1, \ldots, p\}$ for the set of features of interest (FOI) whose effects are decomposed. We write $Z \subseteq \{1, \ldots, p\}$ for the set of split features used to define regions. We use $j \in S$ for an FOI and $z \in Z$ for a split feature. GADGET builds a tree of regions, where each tree node corresponds to a region $A \subseteq \mathcal{X}$, with observation index set $\mathcal{I}_A = \{i : \mathbf{x}^{(i)} \in A\}$. The root region is A_0, with $\mathcal{I}_{A_0} = \{1, \ldots, n\}$. For PD, let $\tilde{\mathbf{x}}_j = \{\tilde{x}_j^{(1)}, \ldots, \tilde{x}_j^{(K_j)}\}$ be the evaluation grid (e.g., quantile-based or equidistant grid points on the feature axis), and let $k = 1, \ldots, K_j$ index its grid points. For ALE, let $\tilde{x}_j^{(0)} < \tilde{x}_j^{(1)} < \cdots < \tilde{x}_j^{(K_j)}$ be the interval boundaries, and let $k = 1, \ldots, K_j$ index the intervals. Let $\mathcal{I}_{j,k} = \{i : x_j^{(i)} \in (\tilde{x}_j^{(k-1)}, \tilde{x}_j^{(k)}]\}$ denote the index set of observations in interval k, and $n_{j,k} = |\mathcal{I}_{j,k}|$ its size; for $k = 1$, the interval also includes its lower boundary $\tilde{x}_j^{(0)} = \min_i x_j^{(i)}$. We write t for a candidate split point on split feature z.

2.1 *Feature effects*

For an FOI j, feature effect methods summarize how the prediction $\hat{f}(\mathbf{x})$ changes with varying x_j-values, while the remaining features $\mathbf{x}_{-j}$ are either held fixed (ICE) or averaged over (PD).

ICE curves (Goldstein et al., 2015) are observation-level effect curves. For a feature subset W, observation i, and grid point $\tilde{\mathbf{x}}_W$, the ICE function fixes $\mathbf{x}_{-W}^{(i)}$ at its observed values and replaces $\mathbf{x}_W^{(i)}$ by $\tilde{\mathbf{x}}_W$:

$$\hat{f}_W^{(i)}(\tilde{\mathbf{x}}_W) = \hat{f}(\tilde{\mathbf{x}}_W, \mathbf{x}_{-W}^{(i)}),$$

where the superscript (i) marks the observation whose $\mathbf{x}_W^{(i)}$ values are replaced by the grid point $\tilde{\mathbf{x}}_W$. For a single feature j, this becomes $\hat{f}_j^{(i)}(\tilde{x}_j) = \hat{f}(\tilde{x}_j, \mathbf{x}_{-j}^{(i)})$. *PD* (Friedman, 2001) averages these ICE values over observations. At the population level, the PD function for W is

$$f_W^{\text{PD}}(\mathbf{x}_W) = \mathbb{E}_{X_{-W}}\left[\hat{f}(\mathbf{x}_W, X_{-W})\right] = \int \hat{f}(\mathbf{x}_W, \mathbf{x}_{-W})\, dP_{X_{-W}}(\mathbf{x}_{-W}).$$

The empirical PD estimate at grid point $\tilde{\mathbf{x}}_W$ is

$$\hat{f}_W^{\text{PD}}(\tilde{\mathbf{x}}_W) = \frac{1}{n}\sum_{i=1}^n \hat{f}(\tilde{\mathbf{x}}_W, \mathbf{x}_{-W}^{(i)}) = \frac{1}{n}\sum_{i=1}^n \hat{f}_W^{(i)}(\tilde{\mathbf{x}}_W).$$

ICE curves reveal heterogeneity that PD averages out. If different observations produce different curve shapes, the global PD curve can hide region-specific patterns (see Figure 2). PD is simple and model-agnostic, but averaging over the marginal distribution of the remaining features can evaluate feature combinations that are rare or implausible when features are dependent. ALE reduces this concern by estimating local prediction changes from the conditional distribution of the remaining features given X_j.

ALE (Apley and Zhu, 2020) partitions the support of a numeric feature X_j into K_j intervals, typically induced by empirical quantiles of X_j, and accumulates finite differences of $\hat{f}$ in each interval. For $i \in \mathcal{I}_{j,k}$, we denote this finite difference by $\hat{f}(\tilde{x}_j^{(k)}, \mathbf{x}_{-j}^{(i)}) - \hat{f}(\tilde{x}_j^{(k-1)}, \mathbf{x}_{-j}^{(i)})$.

The centered empirical ALE curve is

$$\hat{f}_j^{\mathrm{ALE}}(x_j) = \sum_{k=1}^{k(x_j)} \frac{1}{n_{j,k}} \sum_{i \in \mathcal{I}_{j,k}} \left[\hat{f}(\tilde{x}_j^{(k)}, \mathbf{x}_{-j}^{(i)}) - \hat{f}(\tilde{x}_j^{(k-1)}, \mathbf{x}_{-j}^{(i)}) \right] - c_j,$$

where $k(x_j) \in \{1, \ldots, K_j\}$ denotes the index of the interval containing x_j, and c_j is chosen so that $n^{-1} \sum_{i=1}^{n} \hat{f}_j^{\mathrm{ALE}}(x_j^{(i)}) = 0$. Because the local differences are evaluated only at observations whose feature value already falls within each interval, ALE avoids extrapolation into low-density regions and better reflects local changes supported by the observed feature distribution. In **xplaineff**, ALE for categorical features is handled separately through level-wise differences; see Section 4.

2.2 *GADGET algorithm*

With the notation above, the set of FOI S may contain one or more features, and the set of split features Z may differ from S. GADGET starts from a generic local effect function $h(x_j, \mathbf{x}_{-j}^{(i)})$, adopted from Herbinger et al. (2024). It quantifies the local effect of feature j for observation i at a specific value $x_j \in \mathcal{X}_j$, calculated by a function $h : \mathbb{R}^p \to \mathbb{R}$ (e.g., mean-centered ICE values for PD, or local increments for ALE). Within a region A, the heterogeneity of these local effects at x_j is captured by the deviation of h from its regional mean, summarized by a squared-error loss:

$$\mathcal{L}_j(A, x_j) = \mathbb{E}\Big[\big(h(x_j, X_{-j}) - \mathbb{E}[h(x_j, X_{-j}) \mid X \in A]\big)^2 \,\Big|\, X \in A \Big] = \mathrm{Var}\big(h(x_j, X_{-j}) \mid X \in A\big)\,.$$

The expectation is taken over X_{-j} conditional on $X \in A$, so $\mathcal{L}_j(A, x_j)$ is the conditional variance of the local effect at x_j, and it measures the spread of h across observations within the region. We denote by $h_{j,k}^{(i)}$ the observation-level effect value at grid point or interval k for observation i and FOI j. For PD, GADGET subtracts from each ICE value its average over the grid,

$$h_{j,k}^{(i)} = \hat{f}_j^{(i)}(\tilde{x}_j^{(k)}) - \frac{1}{K_j} \sum_{\ell=1}^{K_j} \hat{f}_j^{(i)}(\tilde{x}_j^{(\ell)}), \qquad i = 1, \ldots, n,\ k = 1, \ldots, K_j.$$

For ALE, the corresponding observation-level value is the local increment,

$$h_{j,k}^{(i)} = \hat{f}(\tilde{x}_j^{(k)}, \mathbf{x}_{-j}^{(i)}) - \hat{f}(\tilde{x}_j^{(k-1)}, \mathbf{x}_{-j}^{(i)}), \qquad i \in \mathcal{I}_{j,k}.$$

Let

$$\mathcal{K}_j(A) = \begin{cases} \{1, \ldots, K_j\}, & \text{for PD,} \\ \{k \in \{1, \ldots, K_j\} : \mathcal{I}_A \cap \mathcal{I}_{j,k} \neq \varnothing\}, & \text{for ALE,} \end{cases}$$

be the index set of PD grid points or ALE intervals over which feature j is summed within region A, and let $\mathcal{I}_{j,k}(A)$ denote the observations in region A at grid point or interval k:

$$\mathcal{I}_{j,k}(A) = \begin{cases} \mathcal{I}_A, & \text{for PD,} \\ \mathcal{I}_A \cap \mathcal{I}_{j,k}, & \text{for ALE.} \end{cases}$$

For $|\mathcal{I}_{j,k}(A)| > 0$ and $k \in \mathcal{K}_j(A)$, the empirical conditional mean of $h_{j,k}^{(i)}$ within region A is

$$\bar{h}_{j,k}(A) = \frac{1}{|\mathcal{I}_{j,k}(A)|} \sum_{i \in \mathcal{I}_{j,k}(A)} h_{j,k}^{(i)}.$$

The unnormalized empirical loss of $\mathcal{L}_j(A, x_j)$ at k is

$$\widehat{\mathcal{L}}_j(A, \tilde{x}_j^{(k)}) = \sum_{i \in \mathcal{I}_{j,k}(A)} \left(h_{j,k}^{(i)} - \bar{h}_{j,k}(A) \right)^2.$$

The empirical risk for feature j is the sum of pointwise empirical losses over the relevant grid points or intervals:

$$\mathcal{R}_j(A, \tilde{\mathbf{x}}_j) = \sum_{k \in \mathcal{K}_j(A)} \widehat{\mathcal{L}}_j(A, \tilde{x}_j^{(k)}).$$

When the grid is fixed, we write $\mathcal{R}_j(A)$ as shorthand for $\mathcal{R}_j(A, \tilde{\mathbf{x}}_j)$. The total risk over all FOI is

$$\mathcal{R}_S(A) = \sum_{j \in S} \mathcal{R}_j(A).$$

$\mathcal{L}_j(A, x_j)$ measures the heterogeneity of local feature effects of feature j at a single value x_j. $\mathcal{R}_j(A)$ aggregates this measure across the grid; it measures the heterogeneity of feature j over its full range within region A in one number. GADGET partitions the feature space into regions with little interaction-related heterogeneity, so that within each region the feature effects are approximately homogeneous and can be summarized by a single regional curve.

At region A, let $V_z \subset \mathcal{X}_z$ be a finite set of candidate split values for split feature $z \in Z$. For a numeric split feature z, the candidates are thresholds taken from the observed values of x_z in A (e.g., unique values or quantiles), and a candidate $t \in V_z$ partitions the region into

$$A^L(t, z) = \{\mathbf{x} \in A : x_z \leq t\}, \qquad A^R(t, z) = \{\mathbf{x} \in A : x_z > t\}.$$

Their observation sets are $\mathcal{I}_{A^L}$ and $\mathcal{I}_{A^R}$. When the candidate split is clear from context, we write these children as A^L and A^R. For a categorical split feature with M levels $\ell_1, \ldots, \ell_M$, V_z has a different meaning. For PD trees, $V_z = \{\ell_1, \ldots, \ell_M\}$, and a candidate level ℓ splits $\{\ell\}$ versus the remaining levels. For ALE trees, the levels are first ordered as $\ell_{(1)}, \ldots, \ell_{(M)}$ according to the selected `order_method`, and $V_z = \{\ell_{(1)}, \ldots, \ell_{(M-1)}\}$. A candidate $\ell_{(m)}$ then splits $\{\ell_{(1)}, \ldots, \ell_{(m)}\}$ versus $\{\ell_{(m+1)}, \ldots, \ell_{(M)}\}$. ALE needs an ordering of the levels to form local increments, so its candidates are cumulative level sets. PD has no such ordering requirement, and one-versus-rest keeps the candidate set of size M instead of $2^{M-1} - 1$.

The GADGET split objective is the reduction in total risk after splitting,

$$\Delta\mathcal{R}(A, t, z) = \mathcal{R}_S(A) - \sum_{j \in S} \left[\mathcal{R}_j(A^L) + \mathcal{R}_j(A^R) \right].$$

A candidate split is specified by a split feature $z \in Z$ and a split value $t \in V_z$; the candidate (t, z) is accepted if both children contain at least a prescribed minimum number of observations. Let $(t^*(A), z^*(A))$ denote the accepted candidate that maximizes $\Delta\mathcal{R}(A, t, z)$. The split improvement is reported as a root-normalized value:

$$\text{imp}(A, t, z) = \frac{\Delta\mathcal{R}(A, t, z)}{\mathcal{R}_S(A_0)}.$$

At the root, the selected candidate is accepted when $\text{imp}(A_0, t^*(A_0), z^*(A_0)) \geq \tau$, where $\tau \in (0, 1)$ is a prescribed threshold. For a non-root node A with parent region A_{par}, GADGET accepts the selected candidate when

$$\text{imp}(A, t^*(A), z^*(A)) \geq \tau \cdot \text{imp}(A_{\text{par}}, t^*(A_{\text{par}}), z^*(A_{\text{par}})).$$

Equivalently, each accepted non-root split must deliver at least a fraction τ of the root-normalized improvement delivered by its parent; the root uses the baseline threshold τ. Independently of this criterion, GADGET also stops splitting a node once a prescribed maximum split depth is reached.

The full procedure can be summarized as the following recursive scheme.

1. **Root:** the root region A_0 contains all observations; the PD grid values or ALE local differences are initialized for every feature in S.
2. **Heterogeneity:** for each feature $j \in S$, the region risk $\mathcal{R}_j(A)$ is computed from mean-centered ICE values which are centered by their grid average at each PD grid point or from the corresponding ALE local differences.
3. **Splitting:** the algorithm scans all candidates (t, z) with $z \in Z$ and $t \in V_z$, keeps only those whose two children each contain at least the prescribed minimum number of observations, selects the candidate $(t^*(A), z^*(A))$ that maximizes $\Delta\mathcal{R}(A, t, z)$, and accepts the split only when the root-normalized improvement rule is satisfied and the maximum split depth has not been reached.
4. **Recursion:** region A is split into A^L and A^R according to $(t^*(A), z^*(A))$. Regional PD or ALE values and region risks are evaluated on each child, and the procedure recurses from step 2 on each child until a stopping condition is reached.

The result is a tree of regions. Within each leaf, or any internal node, the feature effects are evaluated on that node's subset of data, yielding *regional* PD or ALE curves that are more homogeneous and easier to interpret. The tree structure itself reveals which variables and split points best separate different effect regions.

In the package implementation, PD and ALE share the same sufficient-statistic risk update (Appendix II.1). For ALE self-feature splits, **xplaineff** applies a bias correction to the raw heterogeneity reduction (Appendix II.5).

3 Related work

We review R and Python packages for PD, ICE, and ALE feature effects, comparing the scope of supported effect types, how effect computations are organized, how categorical ALE levels are ordered, and whether regional decomposition is supported.

In R, **pdp** computes PD and ICE for fitted models via a user-supplied prediction function and supports parallel evaluation (Greenwell, 2017). **ICEbox** focuses on ICE visualization and adds centered ICE and derivative ICE for diagnosing effect heterogeneity (Goldstein et al., 2026). For ALE, the archived **ALEPlot** implemented ALE and PD (Apley, 2018). The maintained package **ale** reimplements ALE with a user-controlled number of intervals, adds bootstrap confidence intervals, and provides ALE-based p-value and effect-size summaries (Okoli, 2025, 2023). For unordered categorical features, the package imposes a single data-driven ordering: each pair of levels is assigned a distance that is summed over all other features, where each numeric feature contributes a Kolmogorov–Smirnov distance and each non-numeric feature contributes a half-L_1 distance, both computed between the two levels' conditional distributions; the resulting $M \times M$ level-by-level distance matrix is embedded in one dimension via classical MDS. Ordered factors keep their supplied level order. **xplaineff**, in contrast, exposes the level-ordering rule as an explicit hyperparameter, with raw, random, MDS, and PCA-based variants. **effectplots** computes PD and ALE through a C++ backend built on **collapse** (Krantz, 2026); it evaluates PD for continuous and discrete features on subsamples, but restricts ALE to continuous features (Mayer, 2025). **iml** wraps fitted models through its `Predictor` class, provides PD, ICE, and ALE alongside permutation feature importance and interaction statistics, and offers `FeatureEffect$new()` for one feature or one feature pair, plus `FeatureEffects$new()` as a convenience wrapper for multiple features (Molnar et al., 2018). For categorical ALE, **iml** orders unordered factor levels by the same data-driven ordering rule (MDS) without a user-facing option. **DALEX** wraps models through its `explainer` interface; PD, ICE, and ALE are computed by the companion package **ingredients** (Biecek, 2018). **ingredients** derives ALE from `ceteris_paribus` profiles: each observation is replicated once per grid value with the FOI set to that value, and the entire expanded dataset is predicted. Local differences are then computed and subsetted to include only observations that naturally fall into each specific grid interval, before being accumulated into the final ALE curve. In contrast, **ale** and **effectplots** compute local differences directly at

interval boundaries without materializing the full profile matrix, which reduces memory cost on large datasets. All of these packages produce global or observation-level (local) effects. None provides regional decomposition of PD or ALE into regions with more homogeneous effects.

Beyond the foundations introduced in Section 2, the Regional Effect Plots with implicit Interaction Detection (REPID) framework (Herbinger et al., 2022) is a special case of the GADGET framework (Herbinger et al., 2024) with a single FOI and identical effect and split feature sets. Gkolemis et al. (2023b, 2026) pursue a related but distinct goal: rather than explaining a fitted model post hoc, they partition the feature space into regions with weak interactions and fit interpretable additive models within each region. The **xplaineff** package implements the PD and ALE variants of the GADGET framework in R.

In Python, `scikit-learn`'s inspection module (Pedregosa et al., 2011) computes PD and ICE on percentile-based grids for both numeric and categorical features but does not implement ALE. Dedicated ALE implementations such as **PyALE** (Jomar, 2024) fill this gap but are limited to global effects. **effector** exposes `RegionalPDP` and `RegionalALE` classes that accept a fitted model and recursively partition the data by minimizing PD- or ALE-based effect heterogeneity, which produces a tree of regional effect curves (Gkolemis et al., 2024). It restricts ALE to numeric features and does not support user-selected categorical factor ordering. For PD, categorical features admit a natural level-by-level summary, so the level-ordering question is specific to categorical ALE.

Among reviewed R packages, **pdp**, **iml**, **DALEX**/**ingredients**, **ale**, and **effectplots** cover global PD, ICE, and ALE effects, but none provides regional decomposition; **xplaineff** fills this gap by implementing the GADGET framework with both PD and ALE variants. In Python, **effector** also offers regional PD and ALE, but restricts ALE to numeric features and does not expose the categorical level-ordering rule as a hyperparameter; **xplaineff** instead makes the ordering method user-selectable. The packages also differ substantially in how they organize model-prediction work, which drives the runtime differences reported in Section 5; Appendix III details the prediction strategies, reuse patterns, parallel backends, and categorical ALE ordering rules for each package (Table 4).

4 An introduction to xplaineff

4.1 Design

The package has two **R6** classes that separate generic tree control from effect-specific computation. `GadgetTree` holds the tree topology and the recursive split logic, exposes `fit()`, `plot()`, `plot_tree_structure()`, and `extract_split_info()`, and delegates effect-specific work to a strategy. The two effect strategies, `PdStrategy` and `AleStrategy`, implement preprocessing, optional node-wise effect transformations, heterogeneity calculation, split search, and the regional-effect plotting that `GadgetTree$plot()` dispatches to. Effect computation, heterogeneity aggregation, and candidate-split evaluation are accelerated in C++ via **Rcpp** and **RcppArmadillo**. Argument validation uses **checkmate** and **cli**, row operations use **data.table**, and plots are drawn with **ggplot2**, **ggraph**, **igraph**, and **patchwork**. **mlr3** learners and fitted R models with standard `predict()` methods work through the package's default prediction path. For models with nonstandard prediction interfaces or outputs, users can supply a `predict_fun` that maps `model` and `newdata` to numeric predictions.

4.2 Inputs and outputs

A typical workflow has two stages: model fitting and regional inspection. Users start from a predictive model that returns numeric predictions, pick an effect strategy (currently `PdStrategy` or `AleStrategy`), and create a `GadgetTree` with controls such as the maximum split depth (`n_split`), minimum node size, improvement threshold (`impr_par`), and split-candidate resolution (`n_quantiles`). Here, split depth is counted along each root-to-leaf path rather than as a global split budget. The `fit()` call takes the reference data, the target name,

a model or precomputed effect object, and two feature-selection arguments: `feature_set` is the FOI set S from Section 2, and `split_feature` is the candidate region-defining set Z. Once the tree is fitted, `extract_split_info()`, `plot_tree_structure()`, and `plot()` return the split table, the tree topology, and the regional effects.

4.3 Example 1: synthetic PD tree

The following self-contained example shows the API shape on synthetic data whose x_2 effect is moderated by x_3.

```
set.seed(1)
n = 500
x1 = runif(n, -1, 1); x2 = runif(n, -1, 1); x3 = runif(n, -1, 1)
y = 0.2 * x1 + ifelse(x3 > 0.3, 3, -3) * x2 + rnorm(n, 0, 0.3)
data = data.frame(x1, x2, x3, y)
model = lm(y ~ x1 + x2 * I(x3 > 0.3), data = data)

tree = GadgetTree$new(
  strategy = PdStrategy$new(), n_split = 2, min_node_size = 50,
  n_quantiles = 40
)
tree$fit(
  data = data, target_feature_name = "y", model = model,
  feature_set = "x2", split_feature = c("x1", "x3"),
  n_grid = 20L
)
split_info = tree$extract_split_info()
print(
  split_info[, c("id", "depth", "n_obs", "node_type",
    "split_feature", "split_value", "int_imp", "is_final")],
  row.names = FALSE,
  digits = 2
)
```

Here `n_quantiles` limits the numeric split candidates to quantile-based thresholds, while `n_grid` sets the PD evaluation grid. `extract_split_info()` returns one row per node; the main columns are shown below. `int_imp` is the root-normalized improvement $\text{imp}(A, t^*(A), z^*(A))$ defined in Section 2.2, and `is_final` marks leaf nodes.

```
#>  id depth n_obs node_type split_feature split_value int_imp is_final
#>   1     1   500      root            x3        0.31    0.99    FALSE
#>   2     2   329      left          <NA>          NA      NA     TRUE
#>   3     2   171     right          <NA>          NA      NA     TRUE
```

The root split recovers the x_3 moderator near its threshold. The two child nodes stay as leaves because the x_2 effect is homogeneous within each x_3 regime, so the tree stops before using the full split depth allowed by `n_split`. The near-one value `int_imp = 0.99` shows that the root split removes essentially all root-level heterogeneity.

4.4 Example 2: bike-sharing PD tree

We return to the bike-sharing data from the introduction. The example draws a random sample from **ISLR2**'s `Bikeshare` data and fits a random forest through **mlr3**. A **xplaineff** tree on top of that model then searches for subregions in which the selected feature effects are more homogeneous. For a compact regional tree, the example uses `hr`, `temp`, `workingday`, and `season` as the FOI set and allows `temp`, `workingday`, and `season` to define regions. The plots below display the regional effect of `hr`.

```
library(xplaineff)
library(mlr3)
library(mlr3learners)
library(ISLR2)

# 1) Load and subsample the Bikeshare data
data("Bikeshare", package = "ISLR2")
set.seed(123)
bike = Bikeshare[sample(seq_len(nrow(Bikeshare)), 1000), ]
factor_features = c("season", "mnth", "weekday", "workingday",
  "holiday", "weathersit")
bike[factor_features] = lapply(bike[factor_features], as.factor)
bike_data = bike[, c("hr", "temp", "workingday", "season",
                     "mnth", "day", "holiday", "weekday",
                     "weathersit", "atemp", "hum", "windspeed",
                     "bikers")]
names(bike_data)[names(bike_data) == "bikers"] = "target"
effect_features = c("hr", "temp", "workingday", "season")
split_features = c("temp", "workingday", "season")

# 2) Fit a black-box regression model with mlr3
task = TaskRegr$new(id = "bike", backend = bike_data, target = "target")
learner = lrn("regr.ranger")
learner$train(task)

# 3) Grow a PD-based GadgetTree on top of the model
tree = GadgetTree$new(
  strategy = PdStrategy$new(),
  n_split = 2,
  min_node_size = 50
)
tree$fit(
  data = bike_data,
  target_feature_name = "target",
  model = learner,
  feature_set = effect_features,
  split_feature = split_features,
  n_grid = 20L
)
```

The fitted tree uses the same accessors as the synthetic example, with the same column meanings.

```
# 4) Inspect the tree structure, splits, and regional PD/ICE curves
tree$plot_tree_structure()
split_info = tree$extract_split_info()
split_info[, c("id", "depth", "n_obs", "node_type",
    "split_feature", "split_value", "int_imp", "is_final")]
```

```
#>  id depth n_obs node_type split_feature split_value int_imp is_final
#>   1     1  1000      root          temp        0.49    0.44    FALSE
#>   2     2   489      left        season           1    0.10    FALSE
#>   3     2   511     right    workingday           1    0.08    FALSE
#>   4     3   224      left          <NA>        <NA>      NA     TRUE
#>   5     3   265     right          <NA>        <NA>      NA     TRUE
#>   6     3   359      left          <NA>        <NA>      NA     TRUE
#>   7     3   152     right          <NA>        <NA>      NA     TRUE
```

The root split separates cooler and warmer observations at `temp = 0.49`. In the cooler branch, the next split uses `season`: level 1 goes to one child, levels 2–4 to the other. In the warmer branch, the next split uses `workingday`: level 1 denotes working days, level 0 non-working days (Appendix I).

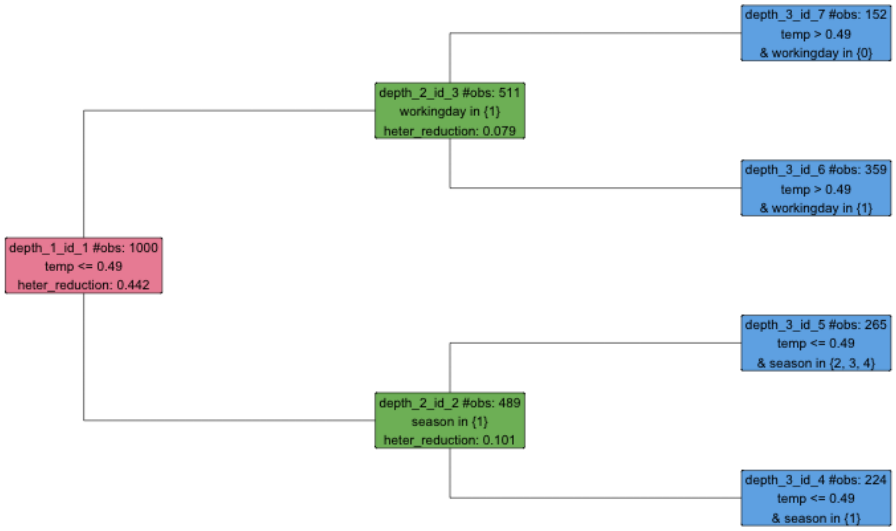

Figure 1: Tree structure for the bike-sharing example. The root split is on `temp`; the two second-level splits refine the cooler branch by `season` and the warmer branch by `workingday`.

We then plot the PD and ICE curves for `hr`. The command returns regional plots for every node; Figure 2 shows just the root and its two children.

```
tree$plot(
  data = bike_data,
  target_feature_name = "target",
  features = c("hr")
)
```

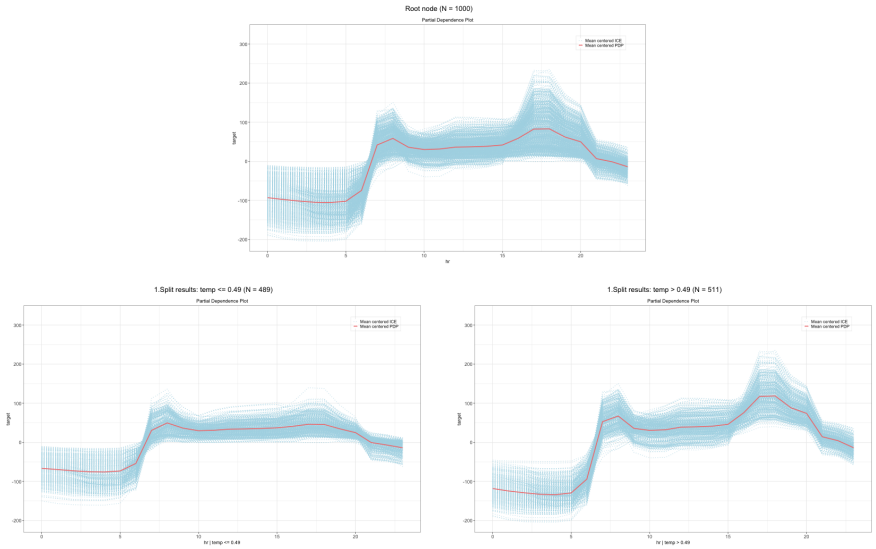

Figure 2: PD and ICE curves for `hr` at the root and in the two child nodes after the root split. The top panel shows the global curve at the root; the bottom-left panel corresponds to `temp <= 0.49`, and the bottom-right panel corresponds to `temp > 0.49`.

At the root, the PD curve averages over all observations and the ICE curves spread widely; the spread points to substantial between-observation heterogeneity. After the split on `temp`, both regions still show low predicted demand at night with morning increases. The warmer branch has a stronger evening peak, and the ICE curves within each branch are more concentrated than at the root. The plots match the split table: the hourly PD pattern is not globally stable, and the tree isolates covariate regions where the heterogeneity is easier to inspect.

4.5 Example 3: bike-sharing ALE tree

We rerun the bike-sharing analysis with `AleStrategy` on the same fitted model, data, FOI set, and candidate split set. The ALE-specific control `n_intervals` sets the number of intervals used for ALE accumulation. The constructor argument `impr_par` is the improvement threshold τ from Section 2.2.

```
# Same data preparation as in Example 2, then:
tree = GadgetTree$new(
  strategy = AleStrategy$new(),
  n_split = 2,
  impr_par = 0.01,
  min_node_size = 50
)
tree$fit(
  data = bike_data,
  target_feature_name = "target",
  model = learner,
  feature_set = effect_features,
  split_feature = split_features,
  n_intervals = 10
)
split_info = tree$extract_split_info()
split_info[, c("id", "depth", "n_obs", "node_type",
    "split_feature", "split_value", "int_imp", "is_final")]

#>  id depth n_obs node_type split_feature split_value int_imp is_final
```

```
#>   1     1  1000      root    workingday           0    0.53    FALSE
#>   2     2   316      left        season           1    0.02    FALSE
#>   3     2   684     right          temp        0.47    0.16    FALSE
#>   4     3    72      left          <NA>        <NA>      NA     TRUE
#>   5     3   244     right          <NA>        <NA>      NA     TRUE
#>   6     3   316      left          <NA>        <NA>      NA     TRUE
#>   7     3   368     right          <NA>        <NA>      NA     TRUE
```

The ALE tree first separates working days from non-working days, with `int_imp = 0.53`; this single split removes more than half of the root heterogeneity. The non-working-day branch is then refined by `season`, and the working-day branch by `temp` at `0.47`. The two trees differ from Example 2: the PD tree splits on `temp` first and uses `season` and `workingday` as second-level refinements, whereas the ALE tree puts `workingday` at the root and refines the child nodes by `season` and `temp`. Regional ALE curves for `hr` come from the same `plot()` call as in Example 2. We use `mean_center = TRUE` to display the accumulated ALE curves on a centered scale, which makes the regional shapes easier to compare (Figure 3).

```
tree$plot(
  data = bike_data,
  target_feature_name = "target",
  features = c("hr"),
  mean_center = TRUE
)
```

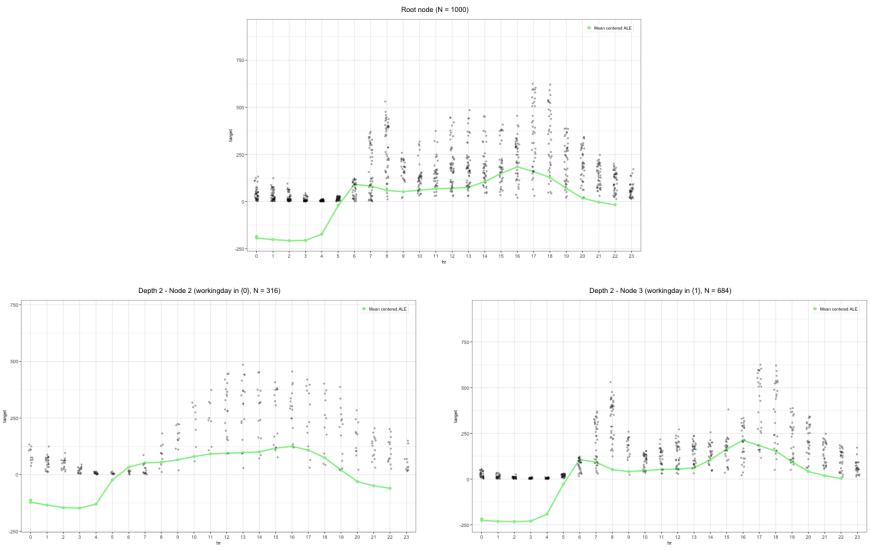

Figure 3: ALE curves for `hr` at the root and in the two child nodes after the root split on `workingday`. The top panel shows the global curve at the root; the bottom-left panel corresponds to non-working days (level 0), and the bottom-right panel to working days (level 1).

The root curve mixes two qualitatively different hourly patterns and is therefore flatter and less interpretable. After the split on `workingday`, the non-working-day branch has a single midday peak typical of leisure demand, while the working-day branch shows the commute pattern with morning and evening peaks. The PD tree in Example 2 splits on `temp` at the root, while the ALE tree picks the working-day contrast that is visible in the displayed `hr` curves.

4.6 PD vs ALE

The split objective in Section 2.2 does not depend on the effect type, but the two bike-sharing trees show that the strategies differ in four places: their accepted inputs, their internal

effect representation, the candidate splits they enumerate on categorical features, and their strategy-specific controls. Table 1 summarizes these differences.

	`PdStrategy`	`AleStrategy`
Accepted input	precomputed PD/ICE object or `model`	`model` and data only
Internal effect representation	mean-centered ICE on the evaluation grid	observation-level local effects with interval sufficient statistics
Candidate splits on categoricals	one-level-versus-rest	cumulative binary level-set splits on a linear order of levels
Strategy-specific controls	`n_grid`	`n_intervals`, `order_method`

Table 1: Differences between the two effect strategies. For `PdStrategy`, precomputed PD/ICE input means an ICE-style effect object, such as an **iml** `FeatureEffect` or `FeatureEffects` object. The ALE path currently uses a model and data because available precomputed ALE objects, including **iml**'s aggregate ALE curves, do not expose the observation-level local effects and interval sufficient statistics required by the ALE split search. The `order_method` argument controls how category levels are placed on a linear order before evaluating cumulative binary level-set splits: `"raw"` keeps the existing order, `"random"` uses a random order, and `"mds"` or `"pca"` embed a matrix of pairwise distances between levels in one dimension.

The bike-sharing example encodes the categorical variables listed in Appendix I as factors before model fitting. More generally, the package converts character split features to factors automatically, but treats numeric binary indicators as numeric unless users encode them as factors before fitting. For ALE-specific controls beyond `n_intervals` and `order_method`, see the package help pages (e.g., `?GadgetTree`, `?AleStrategy`).

5 Benchmark experiments

Table 2 summarizes the benchmark tasks and implementations. The following subsections describe the data-generating mechanisms, package calls, and results.

Table 2: Overview of benchmark tasks, packages, and experimental grids.

Benchmark	Packages or implementations	Object	Benchmark setting*
Global PD/ICE and ALE runtime	**xplaineff**, **iml**, **DALEX**/**ingredients**, **effectplots**; **pdp** (PD only); **ale** (ALE only)	All univariate PD/ICE and ALE effects over the p features.	bagged trees or f_{toy}; $n \in \{1{,}000, 5{,}000, 10{,}000, 20{,}000\}$; $p \in \{10, 20, 50, 100\}$; $K_j \in \{10, 20, 50\}$.
Regional PD and ALE split-search runtime	**xplaineff**, **effector**	Recursive split search after global effect information has been computed.	Same grid as above, plus effect $\in \{\text{PD}, \text{ALE}\}$ and `n_split` $\in \{2, 5, 8, 10\}$.
Categorical level ordering	**xplaineff** ordering rules; exhaustive binary-partition search	Root split recovery when the categorical moderator is fixed as the split feature.	`order_method`: `raw`, `random`, `mds`, `pca`; exhaustive-search reference reported for $M \leq 12$; $\lambda_{\text{leak}} \in \{0, 0.025, 0.05, 0.075, 0.1, 0.15, 0.2, 0.25, 0.5, 1\}$; $M \in \{6, 8, 12, 20\}$; $n \in \{500, 1{,}000, 2{,}000, 5{,}000\}$; $p \in \{3, 10, 25, 50\}$; $\beta_{\max} \in \{1, 2, 4, 8\}$.

* For the runtime rows, K_j is the PD grid size or ALE interval count from Section 2; non-swept base values are introduced below. For the categorical diagnostic, λ_{leak} is the leakage strength, M is the number of levels, and $\beta_{\max}$ is the response-side slope magnitude. These quantities are defined in its data-generating mechanism.

5.1 *Efficiency benchmarks*

We run two runtime benchmarks. The first times the computation of univariate PD/ICE and ALE effects for all p features in R. It compares **xplaineff** with packages that expose the same global effect types: **pdp** (Greenwell, 2017), **iml** (Molnar et al., 2018), **DALEX/ingredients** (Biecek, 2018), **ale** (Okoli, 2025, 2023), and **effectplots** (Mayer, 2025). The second times regional split search after global effect information has been computed. It compares **xplaineff** with the Python package **effector**, which also implements regional PD and regional ALE. Both benchmarks use the same synthetic data-generating mechanism.

Data-generating mechanism. Throughout the benchmark descriptions, n denotes the simulated sample size and p the number of features. We generate synthetic data with $X_j \sim \mathcal{U}(-1, 1)$, $j = 1, \ldots, p$. The deterministic part of the response uses the first three features and is identical for all p:

$$y_{\text{det}} = 5x_1 + 5x_2 + \mathbf{1}(x_3 > 0)\, 10x_1 - \mathbf{1}(x_3 > 0)\, 10x_2.$$

Feature x_3 acts as a moderator: when $x_3 \leq 0$, both x_1 and x_2 have an effect of $+5$; when $x_3 > 0$, the effect of x_1 increases to $+15$ while the effect of x_2 reverses to -5. All features beyond x_1, x_2, x_3 are pure noise, so increasing p grows the split-search space without changing the true signal. Noise is $\varepsilon \sim \mathcal{N}(0, 0.1 \cdot \text{sd}(y_{\text{det}}))$, giving $y = y_{\text{det}} + \varepsilon$.

Prediction settings. Both runtime benchmarks use two prediction settings. The first is a bagged-tree ensemble implemented through random-forest APIs. In this setting, the global feature effect benchmark fits one **ranger** model per data set and passes the same fitted object to all R packages. The regional split-search benchmark uses the same **ranger** setup for **xplaineff** and a matched **scikit-learn** `RandomForestRegressor` for **effector**. Across these runs, we use 100 trees, bootstrap sampling, minimum leaf size 1, seed 21, and single-threaded prediction. By default, **scikit-learn**'s `RandomForestRegressor` uses all features at each split for regression. We therefore set **ranger**'s `mtry = p`, instead of its default $\lfloor \sqrt{p} \rfloor$. With all features available at each split, the fitted ensembles are bagged trees rather than feature-subsampled random forests, and tree diversity comes from bootstrap sampling. The **ranger** model uses variance splitting, `sample.fraction = 1`, and `num.threads = 1`; the **scikit-learn** model uses squared-error splitting, unrestricted tree depth, and `n_jobs = 1`. The two implementations are not bitwise identical because some tree-growing controls do not have direct counterparts. The toy model setting replaces the fitted model by the analytic function $f_{\text{toy}}(x) = y_{\text{det}}(x)$. This makes prediction cost negligible and uses the same deterministic prediction rule on both sides. The bagged-tree panels include model prediction, data construction, aggregation, and output materialization. The toy model panels mainly measure package-side data construction, aggregation, and object construction.

Run environment. All runtime benchmarks were run on the same benchmark server. The server ran Ubuntu 20.04.6 LTS on two Intel Xeon E5-2650 v2 CPUs, with 16 physical cores, 32 hardware threads, and 62 GB of memory. The software environment used R 4.4.1 and Python 3.8.10. The package versions were **xplaineff** 0.1.0, **ranger** 0.16.0, **pdp** 0.8.3, **iml** 0.11.4, **DALEX** 2.5.3, **ingredients** 2.3.0, **ale** 0.5.3, **effectplots** 0.2.2, **data.table** 1.16.0, **scikit-learn** 1.3.2, and **effector** 0.1.6. The benchmark process fixes OpenMP, BLAS, **data.table**, and **RcppParallel** thread counts to one and sets `PYTHONHASHSEED = 21`. Runtime data are generated with seed 21; each timed cell uses one untimed warm-up run to exclude one-time setup costs such as just-in-time compilation and cache population, followed by 20 timed repetitions. Absolute wall-clock times are hardware-dependent, but all comparisons within a panel share the same data, prediction setting, thread configuration, and repetition protocol.

Global feature effect computation. The performance measure is wall-clock time in seconds. For each implementation, we time the univariate effect-computation path used by the

benchmark on the same fitted model and keep the package's native return type. The package-level call paths and runtime controls are summarized in Appendix III. Independent benchmark cells are run in parallel by the outer benchmark runner, but package-internal parallel evaluation is disabled to ensure that each recorded package call uses single-threaded model prediction: **pdp** uses `parallel = FALSE`, **ale** uses `parallel = 0`, and **iml** is run without a **future** parallel plan. All packages use the full dataset without bootstrap resampling: **ingredients** uses `N = n`, **ale** uses `sample_size = n` and `boot_it = 0`, and **effectplots** uses `pd_n = n` or `ale_n = n`. All PD calls use K_j grid points, and all ALE calls use K_j intervals with the same quantile grid when the interface supports this setting. The global R benchmark grid contains three one-dimensional sweeps: it varies $n \in \{1{,}000, 5{,}000, 10{,}000, 20{,}000\}$ at fixed $p = 20$ and $K_j = 20$, varies $p \in \{10, 20, 50, 100\}$ at fixed $n = 10{,}000$ and $K_j = 20$, and varies $K_j \in \{10, 20, 50\}$ at fixed $n = 10{,}000$ and $p = 20$. Figure 4 reports all three sweeps. All cells completed except the p=100 cells of **DALEX/ingredients** (PD and ALE, under both prediction settings), whose runs were repeatedly terminated by the server's out-of-memory killer and skipped on resumption; the corresponding curves therefore end at p=50.

Figure 4 shows median runtimes with interquartile-range ribbons; **xplaineff** is shown with both its default C++ backend and its pure-R computation path. At the common cell n=10,000, p=20, and K_j=20 (also the centroid of the regional benchmark below), **xplaineff** takes 172 s for global PD/ICE and 15 s for global ALE with bagged trees using the R path. The C++ backend is similarly fast, differing by less than 2 s in both cases.

For PD, **pdp**, **DALEX/ingredients**, **iml**, and **effectplots** take 150, 127, 310, and 189 s at this cell, placing **xplaineff** in the middle of the field. This clustering is expected from the pass structure in Table 4: every package scores the same $n \times K_j$ perturbed rows per feature through the same single-threaded model, so the shared prediction cost dominates, and the bagged-tree PD curves rise together along all three sweeps. The exception is **iml**, which stays roughly a factor of two above the rest (871 s versus 278–397 s at n=20,000): it feeds the $n \times K_j$ rows to the model in 1,000-row chunks, 200 prediction calls per feature at the common cell, so per-call prediction overhead accumulates. The toy model panels support this reading. With prediction essentially free, **xplaineff**, **effectplots**, and **pdp** finish every displayed PD cell in under 16 s, while **iml** and **DALEX/ingredients** remain one to two orders of magnitude above them: chunked prediction scheduling (**iml**) and ceteris-paribus profile construction and aggregation (**DALEX/ingredients**) are package-side costs that do not disappear when prediction becomes cheap. Despite this heavy construction overhead, **DALEX/ingredients** is slightly faster than most at the common cell under bagged trees (127 s). Subtracting its toy model time (26.5 s) from its bagged-tree time suggests that its **ranger** prediction calls are roughly 40% faster than other packages, possibly due to differences in how the $n \times K_j$ prediction input is constructed (data type, memory layout, or column ordering). This prediction-time advantage compensates for its heavier construction cost.

The ALE panels split the packages into two regimes. **xplaineff**, **effectplots**, **iml**, and **ale** score $2n$ boundary rows per feature regardless of K_j, whereas **DALEX/ingredients** materializes and scores the full $n \times K_j$ profile before differencing (Table 4). At the common cell, the boundary-pass group needs 15–43 s, while **DALEX/ingredients** takes 534 s, about 35× the **xplaineff** time. The same mechanism shapes its curves: **DALEX/ingredients** is the only package whose ALE runtime rises with K_j (422 to 884 s for $K_j = 10$ to 50); its ALE curve also steepens toward the largest n (1,735 s at n=20,000), and its toy model time at the common cell stays within 16% of its bagged-tree time (448 versus 534 s), so nearly all of its ALE cost is profile construction and aggregation rather than prediction. Within the boundary-pass group, **xplaineff** and **effectplots** are essentially tied across all displayed cells, consistent with both stacking the two interval boundaries into a single $2n$-row pass; **iml**, which scores the same rows in two separate passes, is 1.3× slower at the common cell and reaches the same level by n=20,000. **ale** runs 2.8× slower at the common cell, but its offset is a fixed per-feature setup cost rather than a scaling penalty: its toy model runtime is flat at about 20 s across the entire n and K_j sweeps at p=20, about one second per feature, reaching 104 s at p=100. In the bagged-tree panels this constant dominates at small n and is

progressively amortized, from a 19× gap over **xplaineff** at n=1,000 to below 2× at n=20,000. The two **xplaineff** curves separate only in the toy model panels, where the C++ backend runs up to about a third faster than the R path (0.07 versus 0.11 s for ALE at the common cell); under bagged trees, prediction cost hides this difference entirely.

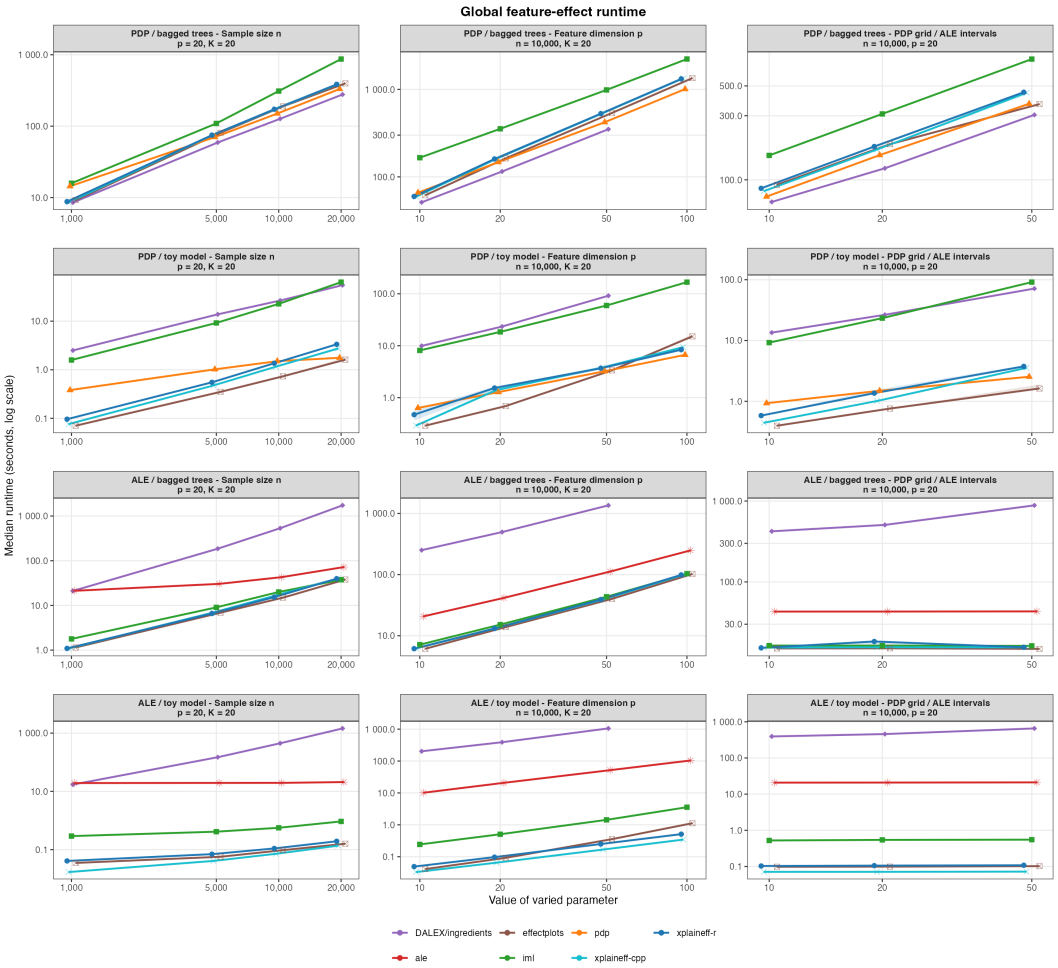

Figure 4: Runtime of global PD/ICE and ALE computation in R. **xplaineff** (default C++ backend and pure-R path) is compared with **pdp**, **iml**, **DALEX/ingredients**, **ale**, and **effectplots**. Rows separate PD/ALE and bagged-tree/toy model settings; columns separate the sweeps over n, p, and K_j. Panel strips report the varied parameter and the fixed values of the two non-varied benchmark parameters. Tree model timings include package-specific prediction batching, while toy model timings mainly measure R-side data construction and aggregation. For **xplaineff**, **pdp**, and **iml**, PD-side timings use ICE profiles from which PD curves can be obtained. Only complete cells are plotted; the **DALEX/ingredients** curves end at p=50 because its p=100 cells did not complete (see text). Each displayed cell is measured after one untimed warm-up run followed by 20 successful timed repetitions. Points and lines show medians; shaded ribbons show the interquartile range.

Regional split-search. For regional split-search, **xplaineff** is compared with **effector** on the same synthetic design. We report two timings: split-search runtime, which starts after the required global PD or ALE information has been computed and measures only regional tree partitioning, and total runtime, which includes both global effect computation and the subsequent regional split-search. Both packages grow binary regional trees with minimum node size 50. The packages are timed under their documented stopping rules rather than at a fixed number of nodes. Unless otherwise stated, all p features are available both as FOI and as candidate split features. The regional grid sweeps four axes through the centroid cell at n=10,000, p=20, $K_j = 20$, and `n_split` = 2: $n \in \{1{,}000, 5{,}000, 10{,}000, 20{,}000\}$, $p \in \{10, 20, 50, 100\}$, $K_j \in \{10, 20, 50\}$, and `n_split` $\in \{2, 5, 8, 10\}$. Here `n_split` caps the

number of splits along any root-to-leaf path of the regional tree, matched to **effector**'s `max_depth`. Each regional cell is measured after one untimed warm-up run followed by 20 timed repetitions. Increasing p and K_j increases the number and cost of candidate split evaluations while leaving the true signal unchanged.

Figures 5 and 6 show the split-search and total-runtime benchmarks against **effector**. In the split-search view, **xplaineff** is faster in every displayed cell. At the common cell n=10,000, p=20, $K_j = 20$, and `n_split` $= 2$, the split-search speed-up is about 35× for regional PD with bagged trees and about 119× for regional ALE with bagged trees. The corresponding total-runtime speed-ups are smaller, about 1.2× for PD and 2.6× for ALE, because total time also includes global feature effect construction. This distinction is especially visible for bagged-tree PD, where global precomputation dominates the total runtime and the total comparison is close to parity along the n sweep. For ALE with bagged trees, the split-search speed-up stays near two orders of magnitude along the n sweep, while the total-runtime speed-up narrows from about 9× at n=1,000 to about 2× at n=20,000 as the global effect computation grows. Along the p and K_j sweeps, both packages spend more time evaluating candidate splits and feature effects, but **xplaineff** retains large split-search speed-ups. The clearest separation occurs along the `n_split` axis: **xplaineff**'s split-search time remains nearly flat because `n_split` is only a maximum-depth cap and the improvement-based stopping rule selects the same shallow tree once further splits no longer reduce heterogeneity enough. This parsimony is also desirable for explanation: when additional splits do not materially improve the regional heterogeneity objective, the shallower tree leaves fewer rules for the analyst to inspect. By contrast, **effector**'s split-search time grows with the allowed depth, reaching speed-ups of about 220× for bagged-tree PD and 566× for bagged-tree ALE. In the toy-model settings, where prediction and global effect construction are cheap, these split-search gains carry through to much larger total-runtime speed-ups.

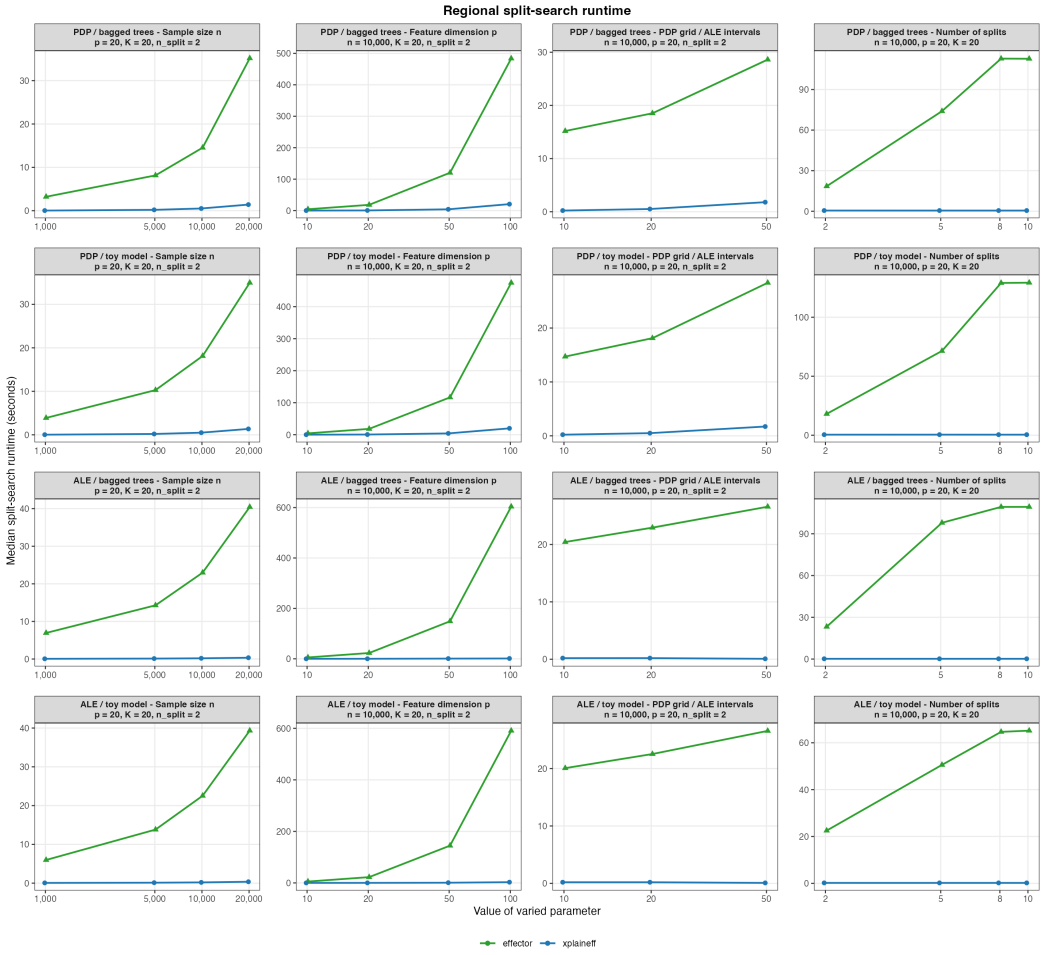

Figure 5: Tree-partitioning runtime (exclude global effect precomputation) for regional PD and ALE. Points and lines show medians, and ribbons show interquartile ranges.

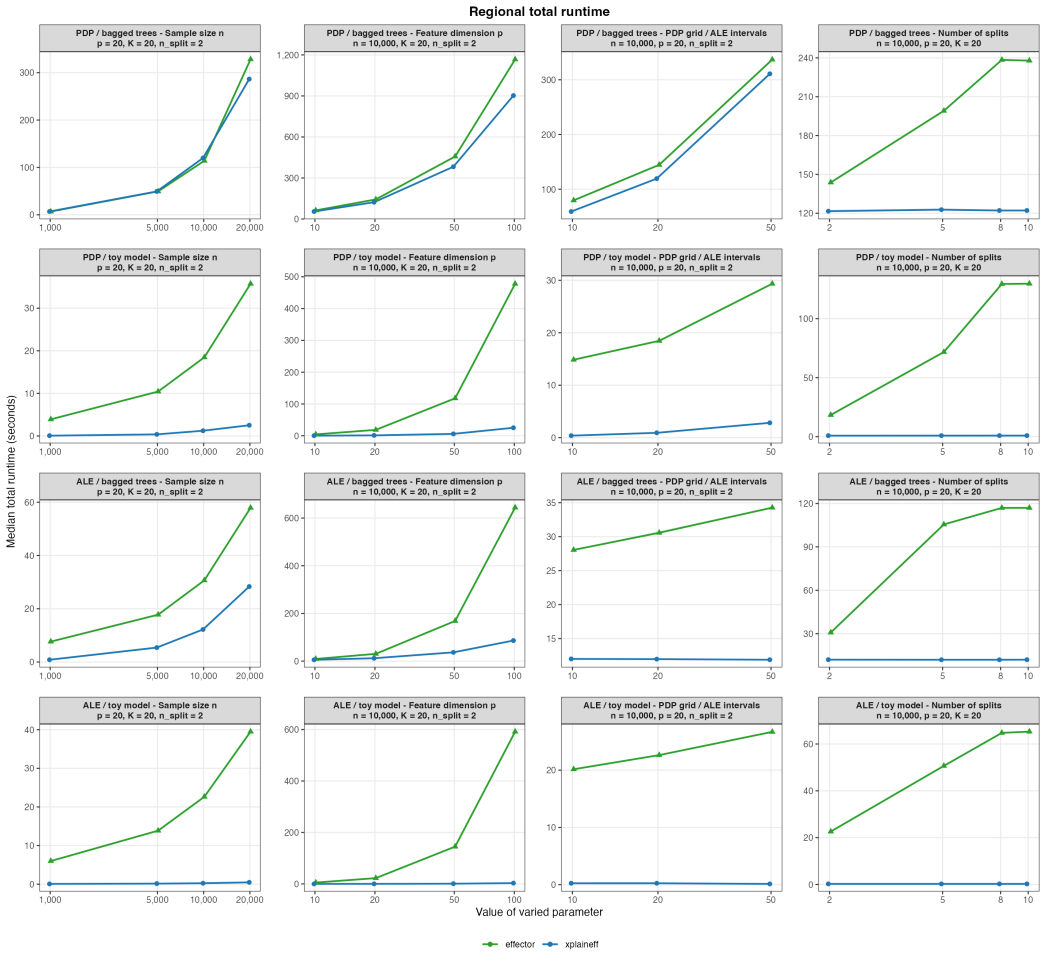

Figure 6: Total runtime (include global effect precomputation) for regional PD and ALE. Points and lines show medians, and ribbons show interquartile ranges.

5.2 *Categorical level-ordering diagnostic*

Aim. This experiment uses a separate synthetic design from the efficiency benchmark above. It varies only the categorical level-ordering rule while keeping the other ALE-tree settings fixed. For an unordered categorical split feature, ALE-tree splits can only cut along the supplied level ordering. Thus, a target level subset is recoverable by the root split only if it is representable as a cumulative cut in that ordering. We fix the categorical split feature as the oracle moderator and vary `order_method` across the four strategies offered by **xplaineff**, together with an exhaustive binary-partition search as an ALE-objective reference. No external package is included for direct comparison: among the reviewed packages in Section 3, none combines a regional split search with a user-selectable categorical level ordering.

Ordering rules. Let $\mathbf{G} \in \mathbb{R}^{M \times M}$ denote the between-level distance matrix: each entry $G_{\ell_a \ell_b}$, $\ell_a, \ell_b \in \{\ell_1, \ldots, \ell_M\}$, sums per-feature contributions across all features other than the focal categorical feature and the response, with each numeric feature contributing a Kolmogorov–Smirnov distance and each non-numeric feature contributing a half-L_1 distance between the two levels' conditional distributions. For a non-numeric feature X_j, this contribution is $\frac{1}{2}\sum_{v \in \mathcal{X}_j} |\widehat{P}(X_j{=}v \mid X_z{=}\ell_a) - \widehat{P}(X_j{=}v \mid X_z{=}\ell_b)|$, with empirical probabilities estimated within each level of the focal categorical split feature X_z.

- **raw** retains the level encoding supplied by the data, with per-seed shuffling so that the encoding cannot exploit a lexical coincidence with the true grouping.

- **random** draws a uniformly random permutation of the levels per seed, giving a signal-independent baseline.
- **MDS** performs classical multidimensional scaling on $\mathbf{G}$: it computes the doubly-centered Gram matrix $B = -\frac{1}{2} J \mathbf{G}^{(2)} J$, where $J = I - M^{-1} \mathbf{1}\mathbf{1}^\top$ is the centering matrix and $\mathbf{G}^{(2)}$ is the elementwise square of $\mathbf{G}$, and orders the levels by the leading eigenvector of B. When the between-level distances are well estimated, this rule gives the same level ordering as **ale** (Okoli, 2025, 2023).
- **PCA** runs principal component analysis directly on the column-centered distance matrix $\mathbf{G}$ (without the elementwise square) and orders the levels by the first principal-component score. This differs from the classical MDS transform $-\frac{1}{2} J \mathbf{G}^{(2)} J$, so the two procedures are not equivalent. They can nevertheless produce similar one-dimensional embeddings when both transformations preserve the same main separation between levels. Our PCA rule is inspired by Wright and König (2019), who order categorical levels by the leading principal component of a level-by-class response probability matrix. Here, the level ordering is part of the candidate-split construction, while the response enters later through the ALE-tree split objective. Using the response for ordering would therefore leak outcome information into the candidate cuts, so we apply the same PCA idea to the level-distance matrix $\mathbf{G}$.
- **exhaustive search** enumerates all $2^{M-1} - 1$ binary partitions of the M levels and selects the partition that maximizes the ALE split objective. In this benchmark we report it only for $M \leq 12$, where the largest cell already enumerates 2,047 partitions; at $M = 20$, the search would require 524,287 candidate partitions.

Data-generating mechanism. We sample $x_2 \sim U(-1, 1)$ as the FOI, draw a categorical feature x_3 with M levels uniformly, and shuffle the level encoding per seed. The benchmark defines a subset of the x_3 levels as signal levels and asks whether the fitted split separates these levels from the remaining levels. We consider two response-generating mechanisms and write y_{det} for the noiseless response. In the *binary-slope* mechanism, we draw the signal level set uniformly from all subsets of size $\lceil M/3 \rceil$. With $g = 1$ for observations whose x_3 level belongs to this set and $g = 0$ otherwise,

$$y_{\text{det}} = \beta_{\max}(2g - 1)x_2.$$

Here, $\beta_{\max}$ is the absolute slope of x_2. In the *group* mechanism, each level ℓ of x_3 receives an independent slope $\beta_\ell \sim U[-\beta_{\max}, \beta_{\max}]$. For an observation with $x_3 = \ell$, the noiseless response is

$$y_{\text{det}} = \beta_\ell x_2.$$

For this mechanism, the signal level set consists of the $\lceil M/3 \rceil$ levels with the largest $|\beta_\ell|$. We set $g = 1$ for observations whose x_3 level belongs to this set and $g = 0$ otherwise. In both mechanisms, Gaussian noise is added as

$$y = y_{\text{det}} + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2), \qquad \sigma_\epsilon = 0.3 \ \text{sd}(y_{\text{det}})$$

where $\text{sd}(y_{\text{det}})$ is computed within each replication. After g has been defined for either mechanism, we add a leakage feature,

$$w = \lambda_{\text{leak}}(2g - 1) + \sqrt{1 - \lambda_{\text{leak}}^2}\,\eta, \qquad \eta \sim \mathcal{N}(0, 1),$$

as a noisy proxy for signal-level membership. In both mechanisms, w is generated from g but does not enter y directly. The parameter $\lambda_{\text{leak}} \in [0, 1]$ controls the signal strength in w; w gives the feature-based ordering rules information about which levels belong together. It is included as a feature, but its information can reach the split search only through the distance matrix $\mathbf{G}$ and the resulting level ordering. The leakage sweep is therefore a controlled diagnostic: λ_{leak} sets how much information about the target level grouping is available

to the ordering rule. We append $p - 3$ independent noise features (alternating Gaussian, uniform, and Bernoulli(0.5)) to reach the target dimensionality. To keep the benchmark focused on level ordering, we use an oracle linear predictor instead of a fitted black-box model. For the binary-slope mechanism, the predictor uses 1, x_2, and $x_2 \cdot g$. For the group mechanism, it uses one $x_2 \cdot \mathbf{1}[x_3 = \ell]$ interaction per level. Thus, failures to recover the level grouping reflect the ordering and split search, not model misspecification.

Experimental design. We sweep $\lambda_{\text{leak}} \in \{0, 0.025, 0.05, 0.075, 0.1, 0.15, 0.2, 0.25, 0.5, 1\}$, $M \in \{6, 8, 12, 20\}$, $n \in \{500, 1000, 2000, 5000\}$, and $p \in \{3, 10, 25, 50\}$. Unless a parameter is varied, we use the base setting $M = 6$, $n = 2000$, $p = 10$, $\lambda_{\text{leak}} = 0.1$, and $\beta_{\max} = 4$. Each ALE tree is restricted to a single root split with x_2 as the FOI, x_3 as the only candidate split feature, $K_j = 20$ ALE intervals, minimum node size 40, and `impr_par` set to 0. We run 30 seeds per cell and assess significance against the `random` ordering baseline via paired two-sided t-tests across seeds within each cell. For each fitted root split, we record six quantities.

1. *Exact recovery*: whether the two child nodes contain the signal level set and its complement, allowing the left and right children to be swapped.
2. *Adjusted Rand index (ARI)*: compares the grouping chosen by the split with the signal-versus-rest reference grouping. A value of 1 means the two groupings are identical; a value near 0 means about as much agreement as a random grouping.
3. *Node accuracy*: the fraction of observations whose x_3 level is assigned to the correct side of the signal-versus-rest split. The left and right children may be swapped.
4. *Interaction importance*: the root-split interaction importance under the ALE objective.
5. *Order correctness*: equals 1 if the m signal levels are exactly the first m or last m levels in the chosen ordering, and 0 otherwise. It checks whether the ordering makes the signal-versus-rest split available to the split search.

In the group mechanism, exact recovery and ARI use the signal level set only as an evaluation reference. Because each level has its own slope, the binary partition with the largest ALE split objective can be different from that reference grouping. The main leakage sweep and the M, n, and p sweeps use the binary-slope mechanism. The group mechanism is evaluated only in a separate λ_{leak} sweep (Appendix IV, Figure 11).

Leakage strength. Figure 7 shows exact recovery and ARI over λ_{leak} for the binary-slope mechanism at the base configuration. `mds` exact recovery rises from 0.13 at $\lambda_{\text{leak}} = 0$ to 0.70 at $\lambda_{\text{leak}} = 0.10$ and reaches 1.00 once $\lambda_{\text{leak}} \geq 0.25$. `pca` tracks `mds` closely, differing by at most one seed in exact recovery and by at most 0.01 in ARI. The `raw` and `random` curves visually overlap at the baseline (0.13 exact recovery and ARI ≈ 0.29) because they do not use the leakage feature w. Both data-driven orderings are significantly above `random` on ARI from $\lambda_{\text{leak}} = 0.075$ onward (paired t-test, $p < 0.001$) and match the exhaustive-search reference once $\lambda_{\text{leak}} \geq 0.25$. The paired-test table for this leakage sweep is reported in Appendix IV.

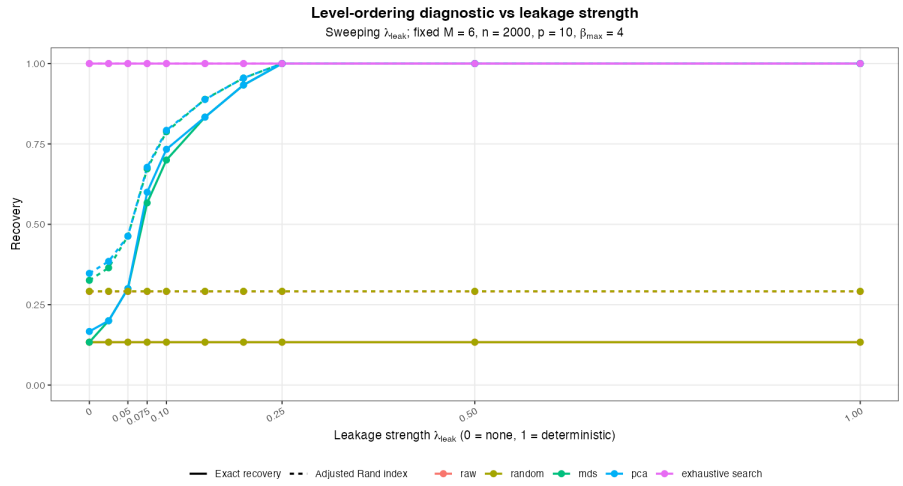

Figure 7: Categorical level-ordering diagnostic across the leakage sweep λ_{leak}, binary-slope mechanism. Solid lines: exact level-set recovery; dashed: adjusted Rand index. Means over 30 seeds at $M = 6$, $n = 2000$, $p = 10$, $\beta_{\max} = 4$. The data-driven orderings `mds` and `pca` rise sharply between $\lambda_{\text{leak}} = 0.05$ and $\lambda_{\text{leak}} = 0.20$ and reach the exhaustive-search reference for $\lambda_{\text{leak}} \geq 0.25$; `raw` and `random` do not use the leakage feature and visually overlap at the baseline.

Cardinality, sample size, and feature dimension. Appendix IV, Figures 8–10, report the M, n, and p sweeps. With M fixed, increasing n gives more observations per x_3 level. Because $\mathbf{G}$ is computed from empirical feature distributions within each level, the part of $\mathbf{G}$ coming from w has less sampling noise. `mds` exact recovery rises from 0.23 at $n = 500$ to 0.83 at $n = 5000$, while the baselines remain low. Increasing p has the opposite effect. Because $\mathbf{G}$ sums distance contributions over the non-focal features, additional noise features dilute the signal from w; `mds` exact recovery falls from 0.90 at $p = 3$ to 0.40 at $p = 50$, although at all four values of p, `mds` and `pca` remain above the `raw` and `random` baselines. The M sweep mainly shows that exact recovery becomes stringent as the number of levels grows. `mds` exact recovery falls from 0.70 at $M = 6$ to zero at $M = 20$, but its ARI remains above the `random` baseline (0.79, 0.62, 0.47, 0.25 versus 0.29, 0.25, 0.24, 0.09).

Group mechanism. Figure 11 reports the separate leakage sweep for the group mechanism. In this mechanism, the reference grouping is the set of levels with the largest $|\beta_\ell|$, whereas exhaustive search selects the binary partition that maximizes the ALE split objective. Because these two groupings need not coincide, exhaustive search still has only 0.10 exact recovery throughout the sweep. We therefore focus on ARI for this sweep. At $\lambda_{\text{leak}} = 0.1$, `mds` reaches ARI 0.27 and `pca` reaches 0.19, compared with 0.09 for `random`. From $\lambda_{\text{leak}} = 0.15$ onward, both data-driven orderings are significantly above `random` on ARI ($p < 0.001$). At $\lambda_{\text{leak}} = 1$, `mds` and `pca` reach ARI 0.52 and 0.41, respectively. Increasing λ_{leak} still improves the data-driven orderings, but the gains are smaller than in the binary-slope mechanism.

6 Discussion and conclusion

We introduced **xplaineff**, an R package for constructing regional feature effect explanations with the GADGET framework. The package provides a common interface for regional PD and ALE trees, separates the generic tree-growing logic from effect-specific computations, and exposes separate controls for the features whose effects are explained and the features allowed to define regions. As a result, users can start from an existing fitted model and obtain the regional-effect tree, the associated split table, and global and regional effect plots in one workflow.

The empirical results show different bottlenecks in the global and regional parts of the workflow. For global PD/ICE and ALE computation, **xplaineff** is competitive with established R workflows. The timing comparison is nevertheless a workflow comparison:

for example, **xplaineff** retains observation-level local effects and interval summaries for downstream regional splitting, whereas some global-effect packages return only aggregated effect curves. In the regional benchmark, the split-search step is substantially faster than the corresponding routines in **effector**. The total-runtime comparison, however, shows that global effect precomputation can dominate when model prediction is expensive, especially for bagged-tree PD. Further optimization effort is therefore most relevant for global effect construction, while parallel split search remains a useful direction for larger trees or broader candidate-search settings.

The categorical level-ordering diagnostic addresses a separate design choice. For unordered factors in ALE trees, the available split candidates depend on the supplied level order. The data-driven `mds` and `pca` orderings are useful but limited heuristics: they can recover useful level groupings when the remaining features carry information about which levels belong together, matching the exhaustive-search reference in the binary-slope diagnostic once the leakage signal is sufficiently strong. The diagnostic also clarifies the limits of exact recovery: as the number of levels grows, ARI remains informative even when exact set recovery becomes stringent, and in the group mechanism the objective-maximizing binary split need not coincide with the evaluation reference grouping.

The current package is best viewed as an explanation tool rather than a surrogate prediction model. After a regional-effect tree has been fitted, **xplaineff** reports the learned partition and the corresponding effect curves, but it does not yet provide a GAM-like prediction rule that maps new observations through the tree and combines regional effects into fitted values. Such an extension would require explicit choices about intercepts, centering, extrapolation outside the effect grids, and how multiple regional effects should be combined. A second direction is to implement additional GADGET variants from Herbinger et al. (2024). In particular, GADGET-SD applies the same decomposition idea to SHAP dependence rather than to PD or ALE effects, and would extend the package to another widely used feature-effect view.

7 Acknowledgments

We thank the **mlr3**, **mlr3misc**, and **iml** developers for tools and conventions that **xplaineff** uses or accepts, and the R Journal editors and reviewers for their feedback. We gratefully acknowledge Shell for sponsoring this project.

8 References

Bibliography

D. Apley. *ALEPlot: Accumulated Local Effects (ALE) Plots and Partial Dependence (PD) Plots*, 2018. URL https://CRAN.R-project.org/package=ALEPlot. R package version 1.1. [p6]

D. W. Apley and J. Zhu. Visualizing the effects of predictor variables in black box supervised learning models. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 82 (4):1059–1086, 2020. doi: 10.1111/rssb.12377. [p1, 2, 3]

P. Biecek. Dalex: Explainers for complex predictive models in r. *Journal of Machine Learning Research*, 19(84):1–5, 2018. URL https://jmlr.org/papers/v19/18-416.html. [p6, 14]

H. Fanaee-T and J. Gama. Event labeling combining ensemble detectors and background knowledge. *Progress in Artificial Intelligence*, 2(2–3):113–127, 2014. doi: 10.1007/s13748-013-0040-3. [p1, 24]

J. H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451. [p1, 3]

V. Gkolemis, T. Dalamagas, E. Ntoutsi, and C. Diou. Rhale: Robust and heterogeneity-aware accumulated local effects. In *ECAI 2023*, volume 372 of *Frontiers in Artificial Intelligence and Applications*, pages 859–866. IOS Press, 2023a. doi: 10.3233/FAIA230354. [p1]

V. Gkolemis, A. Tzerefos, T. Dalamagas, E. Ntoutsi, and C. Diou. Regionally additive models: Explainable-by-design models minimizing feature interactions. *arXiv preprint arXiv:2309.12215*, 2023b. doi: 10.48550/arXiv.2309.12215. [p7]

V. Gkolemis, C. Diou, D. Kyriakopoulos, K. Tsopelas, J. Herbinger, H. Baniecki, D. Rontogiannis, L. Kavouras, M. Muschalik, T. Dalamagas, E. Ntoutsi, B. Bischl, and G. Casalicchio. Effector: A python package for regional explanations. *arXiv preprint arXiv:2404.02629*, 2024. URL https://arxiv.org/abs/2404.02629. [p7]

V. Gkolemis, L. Kavouras, D. Kyriakopoulos, K. Tsopelas, D. Rontogiannis, G. Casalicchio, T. Dalamagas, and C. Diou. Interpretability-by-design with accurate locally additive models and conditional feature effects. *arXiv preprint arXiv:2602.16503*, 2026. doi: 10.48550/arXiv.2602.16503. [p7]

A. Goldstein, A. Kapelner, J. Bleich, and E. Pitkin. Peeking inside the black box: Visualizing statistical learning with plots of individual conditional expectation. *Journal of Computational and Graphical Statistics*, 24(1):44–65, 2015. doi: 10.1080/10618600.2014.907095. [p1, 3]

A. Goldstein, A. Kapelner, and J. Bleich. *ICEbox: Individual Conditional Expectation Plot Toolbox*, 2026. URL https://CRAN.R-project.org/package=ICEbox. R package version 1.2. [p6]

B. M. Greenwell. pdp: An r package for constructing partial dependence plots. *The R Journal*, 9(1):421–436, 2017. doi: 10.32614/RJ-2017-016. [p6, 14]

T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2nd edition, 2009. doi: 10.1007/978-0-387-84858-7. [p1]

J. Herbinger, B. Bischl, and G. Casalicchio. REPID: Regional effect plots with implicit interaction detection. In *Proceedings of the 25th International Conference on Artificial Intelligence and Statistics*, volume 151, pages 10209–10233. PMLR, 2022. URL https://proceedings.mlr.press/v151/herbinger22a.html. [p7]

J. Herbinger, M. N. Wright, T. Nagler, B. Bischl, and G. Casalicchio. Decomposing global feature effects based on feature interactions. *Journal of Machine Learning Research*, 25 (23-0699):1–65, 2024. URL https://jmlr.org/papers/volume25/23-0699/23-0699.pdf. [p2, 4, 7, 22]

G. James, D. Witten, T. Hastie, and R. Tibshirani. *ISLR2: Introduction to Statistical Learning, Second Edition*, 2022. URL https://CRAN.R-project.org/package=ISLR2. R package version 1.3-2. [p1, 24]

D. Jomar. PyALE: Accumulated local effects (ale) plots in Python, 2024. URL https://github.com/DanaJomar/PyALE. Python package version 1.2.0. [p7]

S. Krantz. collapse: Advanced and fast statistical computing and data transformation in R. *Journal of Statistical Software*, 116(1):1–38, 2026. doi: 10.18637/jss.v116.i01. [p6]

M. Mayer. *effectplots: Effect Plots*, 2025. URL https://CRAN.R-project.org/package=effectplots. R package version 0.2.2. [p6, 14]

C. Molnar. *Interpretable Machine Learning: A Guide for Making Black-Box Models Explainable*. Lulu.com, 2019. URL https://christophm.github.io/interpretable-ml-book/. [p1]

C. Molnar, G. Casalicchio, and B. Bischl. iml: An r package for interpretable machine learning. *Journal of Open Source Software*, 3(26):786, 2018. doi: 10.21105/joss.00786. [p6, 14]

C. Okoli. Statistical inference using machine learning and classical techniques based on accumulated local effects (ale). *arXiv preprint arXiv:2310.09877*, 2023. doi: 10.48550/arXiv.2310.09877. [p6, 14, 19]

C. Okoli. *ale: Interpretable Machine Learning and Statistical Inference with Accumulated Local Effects (ALE)*, 2025. URL https://CRAN.R-project.org/package=ale. R package version 0.5.3. [p6, 14, 19]

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. [p7]

M. N. Wright and I. R. König. Splitting on categorical predictors in random forests. *PeerJ*, 7: e6339, 2019. URL https://doi.org/10.7717/peerj.6339. [p19]

Zizheng Zhang
Ludwig-Maximilians-Universität München
zizheng.zhang@stat.uni-muenchen.de

I Appendix: bike-sharing data description

The running example uses the hourly Capital Bikeshare data of Fanaee-T and Gama (2014), distributed as the `Bikeshare` data set in **ISLR2** (James et al., 2022). Each row records one hour, and the response `bikers` counts the total number of bike rentals in that hour. The examples in Section 4.4 draw a random sample of 1000 hours and rename `bikers` to `target`. Table 3 lists the features used to fit the model and to define regions.

Table 3: Features of the `Bikeshare` data used in the running example.

Feature	Type	Description
`hr`	integer	Hour of the day (0–23).
`temp`	numeric	Normalized temperature in $[0, 1]$.
`atemp`	numeric	Normalized feeling temperature in $[0, 1]$.
`hum`	numeric	Normalized humidity in $[0, 1]$.
`windspeed`	numeric	Normalized wind speed in $[0, 1]$.
`season`	factor	Season (1: winter, 2: spring, 3: summer, 4: fall).
`mnth`	factor	Month of the year.
`weekday`	factor	Day of the week.
`workingday`	factor	1 on working days, 0 otherwise.
`holiday`	factor	1 on holidays, 0 otherwise.
`weathersit`	factor	Weather situation, from clear to heavy precipitation.
`day`	integer	Day index within the data set.
`bikers`	integer	Hourly bike rental count (response, renamed `target`).

II Appendix: bias correction for ALE self-feature splits

II.1 Shared sufficient-statistic form of the region risk

Following the notation in Section 2, consider one region A, one candidate split feature $z \in Z$, and one candidate split value $t \in V_z$. The split improvement defined in Section 2.2 is

$$\Delta\mathcal{R}(A, t, z) = \mathcal{R}_S(A) - \sum_{j \in S} \left[\mathcal{R}_j(A^L) + \mathcal{R}_j(A^R)\right],$$

and maximizing $\Delta\mathcal{R}(A, t, z)$ is equivalent to minimizing the total child risk over all candidate (t, z). The fast split search avoids recomputing these child risks from scratch by carrying a

shared sufficient-statistic representation, used by both PD and ALE. The difference between PD and ALE enters only through the definition of the observation-level value $h_{j,k}^{(i)}$ and through which observations enter each grid point or interval; the bookkeeping below is the same in both cases.

For region A, FOI j, and grid point or ALE interval k, define

$$n_{A,j,k} = |\mathcal{I}_{j,k}(A)|, \qquad \mathcal{S}_{A,j,k} = \sum_{i \in \mathcal{I}_{j,k}(A)} h_{j,k}^{(i)}, \qquad \mathcal{Q}_{A,j,k} = \sum_{i \in \mathcal{I}_{j,k}(A)} \left(h_{j,k}^{(i)}\right)^2.$$

For PD, every observation enters every grid point, so $\mathcal{I}_{j,k}(A) = \mathcal{I}_A$. For ALE, only observations whose feature value lies in interval k enter, so $\mathcal{I}_{j,k}(A) = \mathcal{I}_A \cap \mathcal{I}_{j,k}$. The within-group sum of squares is then

$$\sum_{i \in \mathcal{I}_{j,k}(A)} \left(h_{j,k}^{(i)} - \bar{h}_{j,k}(A)\right)^2 = \mathcal{Q}_{A,j,k} - \frac{\mathcal{S}_{A,j,k}^2}{n_{A,j,k}}, \qquad n_{A,j,k} > 0,$$

and the region risk is

$$\mathcal{R}_j(A) = \sum_{k:\, n_{A,j,k} > 0} \left(\mathcal{Q}_{A,j,k} - \frac{\mathcal{S}_{A,j,k}^2}{n_{A,j,k}} \right).$$

The same definitions apply to the children A^L and A^R with A replaced by the corresponding region.

These statistics are additive: when a candidate split partitions A into A^L and A^R, the right-child statistics follow by subtraction,

$$n_{A^R,j,k} = n_{A,j,k} - n_{A^L,j,k}, \qquad \mathcal{S}_{A^R,j,k} = \mathcal{S}_{A,j,k} - \mathcal{S}_{A^L,j,k}, \qquad \mathcal{Q}_{A^R,j,k} = \mathcal{Q}_{A,j,k} - \mathcal{Q}_{A^L,j,k}.$$

After sorting observations by a split feature, a sweep over candidate split values updates the left-child counts, sums, and sums of squares incrementally, obtains the right-child statistics by subtraction, and evaluates $\mathcal{R}_j(A^L) + \mathcal{R}_j(A^R)$ without recomputing variances from scratch.

II.2 ALE self-feature risk decomposition

When the split feature z is itself an FOI, GADGET must also evaluate the ALE risk $\mathcal{R}_z(A^L) + \mathcal{R}_z(A^R)$ for feature z at the children of A. In the implementation, candidate splits do not re-quantize ALE intervals for every t; the current region already carries interval labels and sufficient statistics for the local effects, and each candidate split only repartitions those fixed labels into the left and right children. The crossed-interval derivation below concerns numeric ALE self-feature splits. Categorical ALE uses ordered prefixes of levels rather than numeric thresholds, so this crossed-interval argument does not apply there.

To isolate the contribution of the split feature, we separate its risk from the remaining FOI:

$$\sum_{j \in S} \left[\mathcal{R}_j(A^L) + \mathcal{R}_j(A^R)\right] = \mathcal{R}_{S\setminus\{z\}}(A^L) + \mathcal{R}_{S\setminus\{z\}}(A^R) + \mathcal{R}_z(A) - \Delta_{\text{raw}}(t, z),$$

where $\mathcal{R}_{S\setminus\{z\}}$ follows the convention of $\mathcal{R}_S$ in Section 2.2, summed over all FOI except the split feature z, and the raw self gain is

$$\Delta_{\text{raw}}(t, z) = \mathcal{R}_z(A) - \mathcal{R}_z(A^L) - \mathcal{R}_z(A^R).$$

Since $\mathcal{R}_z(A)$ does not depend on t, ranking candidate splits by the total child risk is equivalent to ranking by

$$\mathcal{R}_{S\setminus\{z\}}(A^L) + \mathcal{R}_{S\setminus\{z\}}(A^R) - \Delta_{\text{raw}}(t, z).$$

The crossed-interval formulas below assume that both children contain variation in the split feature. When the split feature is constant in a child, its self risk in that child is set to zero, since the self ALE contribution is undefined there. This constant-child case is handled separately from the crossed-interval correction.

II.3 Exact raw self gain inside one crossed-interval

For numeric feature z, sort the observations in A by x_z. Because ALE intervals are intervals on the same feature axis, observations from the same interval are contiguous in the sorted order. Hence any split value t can cut at most one ALE interval. All other intervals lie entirely in A^L or entirely in A^R.

Let k^* index the unique crossed-interval, if one exists. Write $\mathcal{I}_{A,z,k^*}$ for the observations in region A whose x_z falls in interval k^*, $\mathcal{I}^L_{A,z,k^*} = \mathcal{I}_{A^L} \cap \mathcal{I}_{A,z,k^*}$, and $\mathcal{I}^R_{A,z,k^*} = \mathcal{I}_{A^R} \cap \mathcal{I}_{A,z,k^*}$, with sizes n_{A,z,k^*}, n^L_{A,z,k^*}, and n^R_{A,z,k^*}, respectively. The following formulas assume $n^L_{A,z,k^*} > 0$ and $n^R_{A,z,k^*} > 0$. Let $h^{(i)}_{z,k^*}$ be the local ALE effect for observation $i \in \mathcal{I}_{A,z,k^*}$, and define the means $\bar{h}_{A,z,k^*}$, $\bar{h}^L_{A,z,k^*}$, and $\bar{h}^R_{A,z,k^*}$ over $\mathcal{I}_{A,z,k^*}$, $\mathcal{I}^L_{A,z,k^*}$, and $\mathcal{I}^R_{A,z,k^*}$. Denote the corresponding within-group sums of squares by SSE_{A,z,k^*}, SSE^L_{A,z,k^*}, and SSE^R_{A,z,k^*}.

All intervals except interval k^* cancel in $\Delta_{\mathrm{raw}}(t,z)$, so

$$\Delta_{\mathrm{raw}}(t,z) = \mathrm{SSE}_{A,z,k^*} - \mathrm{SSE}^L_{A,z,k^*} - \mathrm{SSE}^R_{A,z,k^*}.$$

Applying the standard ANOVA decomposition to the two groups $\mathcal{I}^L_{A,z,k^*}$ and $\mathcal{I}^R_{A,z,k^*}$ gives

$$\mathrm{SSE}_{A,z,k^*} = \mathrm{SSE}^L_{A,z,k^*} + \mathrm{SSE}^R_{A,z,k^*} + \frac{n^L_{A,z,k^*} n^R_{A,z,k^*}}{n_{A,z,k^*}} \left(\bar{h}^L_{A,z,k^*} - \bar{h}^R_{A,z,k^*}\right)^2.$$

Therefore

$$\Delta_{\mathrm{raw}}(t,z) = \frac{n^L_{A,z,k^*} n^R_{A,z,k^*}}{n_{A,z,k^*}} \left(\bar{h}^L_{A,z,k^*} - \bar{h}^R_{A,z,k^*}\right)^2 \geq 0.$$

If t falls exactly on an interval boundary and both children remain nonconstant in z, no interval is crossed and the crossed-interval contribution is zero. If the same boundary split creates a constant-child case, the zero self-risk convention applies instead.

II.4 Null expectation of the raw self gain

Assume now that t is a fixed candidate split crossing interval k^*, and condition on the group sizes n^L_{A,z,k^*} and n^R_{A,z,k^*}. Under a null model with no systematic difference between the two sides of the split, the local ALE effects in the crossed-interval are exchangeable between left and right. Conditional on the group sizes, we model them as independent draws with common mean and variance

$$\mathbb{E}[h^{(i)}_{z,k^*} \mid i \in \mathcal{I}_{A,z,k^*}] = \mu_{A,z,k^*}, \qquad \mathrm{Var}(h^{(i)}_{z,k^*} \mid i \in \mathcal{I}_{A,z,k^*}) = \sigma^2_{A,z,k^*} < \infty.$$

Under the null model, the two subgroup means have the same expectation, and

$$\mathrm{Var}\left(\bar{h}^L_{A,z,k^*} - \bar{h}^R_{A,z,k^*}\right) = \frac{\sigma^2_{A,z,k^*}}{n^L_{A,z,k^*}} + \frac{\sigma^2_{A,z,k^*}}{n^R_{A,z,k^*}} = \sigma^2_{A,z,k^*} \frac{n_{A,z,k^*}}{n^L_{A,z,k^*} n^R_{A,z,k^*}}.$$

Hence

$$\mathbb{E}\left[\left(\bar{h}^L_{A,z,k^*} - \bar{h}^R_{A,z,k^*}\right)^2\right] = \sigma^2_{A,z,k^*} \frac{n_{A,z,k^*}}{n^L_{A,z,k^*} n^R_{A,z,k^*}},$$

and substituting into the exact formula yields

$$\mathbb{E}[\Delta_{\mathrm{raw}}(t,z)] = \sigma^2_{A,z,k^*}.$$

This identity shows that the raw self term $\Delta_{\text{raw}}(t, z)$ has an expected value of one within-interval variance unit purely from sampling noise, with no systematic effect heterogeneity in the interval. Using Δ_{raw} directly as a split score therefore favors splits that merely cut an interval, even when both sides carry the same underlying effect. The bias is interval-local and $O(1)$ on the scale of the crossed-interval, so the correction below subtracts this expected unit directly rather than appealing to its asymptotic negligibility relative to the total region heterogeneity. The expectation is for a pre-specified candidate t, not for the maximum over all candidate split values.

II.5 Bias-corrected split criterion

One simplification for ALE self splits is to rank candidate split values for feature z only by $\mathcal{R}_{S\setminus\{z\}}(A^L) + \mathcal{R}_{S\setminus\{z\}}(A^R)$, the child risk of all FOI except the split feature itself. This avoids the undefined self-ALE term in constant-child cases, but it also drops all information about heterogeneity in the z-effect. In particular, if $S = \{z\}$, then $\mathcal{R}_{S\setminus\{z\}}(A^L) + \mathcal{R}_{S\setminus\{z\}}(A^R) \equiv 0$, so all candidate split values tie.

Section II.4 shows $\mathbb{E}[\Delta_{\text{raw}}(t, z)] = \sigma^2_{A,z,k^*}$ for a fixed crossed-interval candidate t; we subtract an estimate of this expected variance from Δ_{raw}. The unbiased sample variance is

$$\hat{\sigma}^2_{A,z,k^*} = \frac{\text{SSE}_{A,z,k^*}}{n_{A,z,k^*} - 1}, \qquad n_{A,z,k^*} > 1,$$

and the corrected self gain before truncation is

$$\widetilde{\Delta}(t, z) = \Delta_{\text{raw}}(t, z) - \lambda_{\text{bias}} \hat{\sigma}^2_{A,z,k^*},$$

with $\mathbb{E}[\widetilde{\Delta}(t, z)] = (1 - \lambda_{\text{bias}})\sigma^2_{A,z,k^*}$ under the same null model. Setting $\lambda_{\text{bias}} = 1$ centers $\widetilde{\Delta}$ at zero; this is the value used in the implementation. Truncating at zero gives the corrected self gain

$$\Delta_{\text{corr}}(t, z) = \max\{0,\ \widetilde{\Delta}(t, z)\}.$$

Negative values of $\widetilde{\Delta}(t, z)$ mean the raw self gain is smaller than the expected within-interval noise; the truncation treats such cases as zero.

If t lies on an interval boundary and both children remain nonconstant in z, no crossed-interval exists and $\Delta_{\text{corr}}(t, z) = 0$. If the split creates a constant-child case, the zero self-risk convention applies instead of the crossed-interval variance subtraction. If $S = \{z\}$, then $\mathcal{R}_{S\setminus\{z\}}(A^L) + \mathcal{R}_{S\setminus\{z\}}(A^R) \equiv 0$, and the split with the largest Δ_{corr} is selected instead of all candidates tying.

We rank candidate splits by the following corrected child-risk criterion:

$$\mathcal{R}^{\text{corr}}(t, z) = \mathcal{R}_{S\setminus\{z\}}(A^L) + \mathcal{R}_{S\setminus\{z\}}(A^R) - \Delta_{\text{corr}}(t, z).$$

Minimizing $\mathcal{R}^{\text{corr}}(t, z)$ is equivalent to maximizing the corresponding corrected reduction. The correction affects the split-selection ranking; after a split is selected, the stored child objectives and the reported improvement are computed from the child heterogeneity values $\mathcal{R}_j(A^L)$ and $\mathcal{R}_j(A^R)$, with the self risk set to zero in constant-child cases (Section II.2).

II.6 Computational consequence

The correction does not change the sweep complexity. At a candidate split value t, the sweep already maintains the left and right self risks needed for $\Delta_{\text{raw}}(t, z)$. If t cuts interval k^*, the variance estimate $\hat{\sigma}^2_{A,z,k^*}$ is obtained from the parent interval count, sum, and sum-of-squares sufficient statistics. Therefore the correction adds only $O(1)$ work per candidate split value. For one sorted sweep over a single split feature at node A, maintaining risks for all FOI costs $O(|S|\,|\mathcal{I}_A|)$, excluding sorting and candidate construction. Scanning all split features gives $O(|Z|\,|S|\,|\mathcal{I}_A|)$ sweep work at the node, and the bias correction does not change this

asymptotic order.

III Appendix: computation of global effects in the compared packages

Table 4 lists the prediction strategy for PD and ALE, the intermediate state reused across features, the available parallel backend, and the categorical ALE level-ordering rule for each benchmarked package. Throughout, a feature has n observations and K_j grid points (for PD) or intervals (for ALE), following the notation of Section 2.

The *Reused across features* column of Table 4 records what a package computes once and reuses for every feature, instead of rebuilding it on each feature's call. **xplaineff** builds the evaluation grid for every feature once before the feature loop: for PD, the K_j quantile-based grid points for feature j; for ALE, the K_j interval boundaries. It also pre-allocates a single prediction input table, shared across all features: for PD, this table has $n \times K_j$ rows per feature; for ALE, $2n$ rows, with the first n rows holding the lower interval boundaries and the last n rows holding the upper. In both cases, only the focal column is overwritten per feature and restored after prediction; the remaining $p - 1$ columns stay at their observed values throughout. **iml** and **DALEX** instead reuse only a model wrapper, i.e. a `Predictor` or `explainer` object that stores the model, data, and prediction function, while **pdp**, **effectplots**, and **ale** build their prediction inputs anew for every feature.

The *PD* and *ALE* columns describe, for one feature, how the package calls the model's prediction routine and how many rows it scores in total. A *single pass* scores all $n \times K_j$ perturbed rows in one call, as **xplaineff**, **effectplots**, **DALEX/ingredients**, and **effector** do for PD. When **effector**'s `centering` option is on, a preliminary pass of $n \times K_j$ rows first computes the normalization constant, so the total rises to two passes; our benchmark calls **effector** with `centering = True`. **pdp** instead makes one *pass per grid point* (K_j calls of n rows each), and **iml** splits the same $n \times K_j$ rows into fixed *chunks* of 1000 rows, one pass per chunk. For ALE, each interval contributes a difference between predictions at its upper and lower boundary. **xplaineff** and **effectplots** stack both boundaries into one *single stacked pass* over $2n$ rows, whereas **iml**, **ale**, and **effector** evaluate the two boundaries in *two separate passes*. **DALEX/ingredients** takes a different route: it *materializes* the full $n \times K_j$ ceteris-paribus profile—every observation replicated at every grid value—and scores all $n \times K_j$ rows before differencing, which dominates its ALE cost on large data.

The *Parallel* column lists each package's parallel backend, which is disabled in our timings (Section 5) so that each recorded call uses a single thread; the *Cat. ALE order* column records how ALE-capable packages order unordered categorical levels: **xplaineff** exposes this as a user-selectable hyperparameter, **iml** and **ale** impose a fixed data-driven ordering via one-dimensional MDS, **DALEX/ingredients** keeps levels in sorted order, and **effectplots** and **effector** support ALE for numeric features only.

IV Appendix: categorical level-ordering diagnostic

This appendix complements Section 5.2 with the three sweep figures omitted from the main text and a summary of the paired t-test results against the `random` baseline. The leakage sweep at base configuration is shown in Figure 7 of the main text. Figures 8–10 use the binary-slope mechanism. Figure 11 reports the separate leakage sweep for the group mechanism. Mean values are taken over 30 seeds per cell; the $M \leq 12$ cells include the exhaustive binary-partition search, $M = 20$ does not because the 2^{19} candidate space is intractable.

Paired t-tests. Table 5 gives the paired two-sided ARI tests for the binary-slope leakage sweep. For each value of λ_{leak}, `mds` and `pca` are compared with the same-seed `random` baseline over 30 seeds. The table reports the mean ARI difference, the t statistic, and the p-value.

Table 4: Computation of global PD and ALE effects in the compared packages. For one feature, n is the number of observations and K_j the number of PD grid points or ALE intervals (Section 2); a *pass* is one call to the model's prediction routine, and the *PD* and *ALE* columns give the pass structure with the total number of rows scored per feature. Versions: **xplaineff** 0.1.0, **pdp** 0.8.3, **iml** 0.11.4, **DALEX** 2.5.3 / **ingredients** 2.3.0, **effectplots** 0.2.2, **ale** 0.5.3, **effector** 0.1.6.

		Prediction passes per feature			
Package	Reused across features	PD	ALE	Parallel	Cat. ALE order
xplaineff	Evaluation grids and one prediction-input table	Single pass ($n \times K_j$)	Single stacked pass ($2n$)	—	User-selectable (`raw`, `random`, `mds`, `pca`)
pdp	None	Pass per grid point (K_j passes)	—	**foreach**	—
iml	Model wrapper (`Predictor`)	Chunked pass ($n \times K_j$; 1000-row chunks)	Two boundary passes ($2n$)	**future**	Data-driven (MDS)
DALEX/ ingredients	Model wrapper (`explainer`)	Single pass ($n \times K_j$)	Materialized profile ($n \times K_j$)	—	Fixed (sorted levels)
effectplots	None	Single pass ($n \times K_j$)	Single stacked pass ($2n$)	—	Numeric only
ale	None	—	Two boundary passes ($2n$)	**furrr**	Data-driven (MDS)
effector	Precomputed `PDP/ALE` object per feature, reused in regional split search	One pass ($n \times K_j$); two passes when `centering=True`	Two boundary passes ($2n$)	—	Numeric only

Table 5: Paired two-sided t-tests of ARI for `mds` and `pca` versus the `random` baseline across the leakage sweep, binary-slope mechanism, 30 seeds per cell. Mean differences are reported on the ARI scale.

	`mds` vs `random`			`pca` vs `random`		
λ_{leak}	ΔARI	t	p	ΔARI	t	p
0	0.034	0.43	0.672	0.056	0.68	0.502
0.025	0.073	0.86	0.399	0.093	1.09	0.286
0.05	0.171	1.93	0.063	0.171	1.93	0.063
0.075	0.381	4.12	< 0.001	0.386	4.40	< 0.001
0.10	0.496	5.94	< 0.001	0.501	6.39	< 0.001
0.15	0.597	8.81	< 0.001	0.596	8.77	< 0.001
0.20	0.663	10.68	< 0.001	0.663	10.68	< 0.001
0.25	0.708	11.19	< 0.001	0.708	11.19	< 0.001
0.50	0.708	11.19	< 0.001	0.708	11.19	< 0.001
1.00	0.708	11.19	< 0.001	0.708	11.19	< 0.001

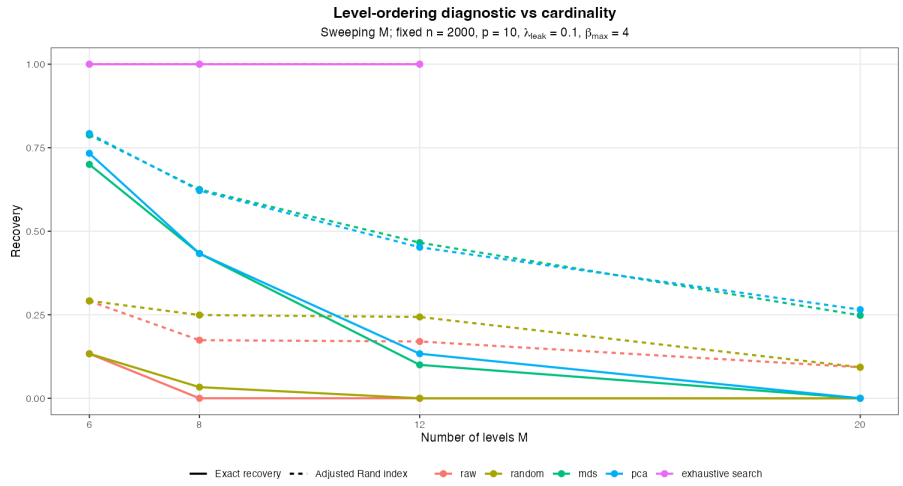

Figure 8: Cardinality sweep $M \in \{6, 8, 12, 20\}$ at $n = 2000$, $p = 10$, $\lambda_{\text{leak}} = 0.1$, $\beta_{\max} = 4$, binary-slope mechanism. Exact recovery becomes much stricter as M grows: `mds` and `pca` exact recovery falls to zero at $M = 20$ while their ARI remains significantly above the `random` baseline. The exhaustive-search reference is shown for $M \leq 12$.

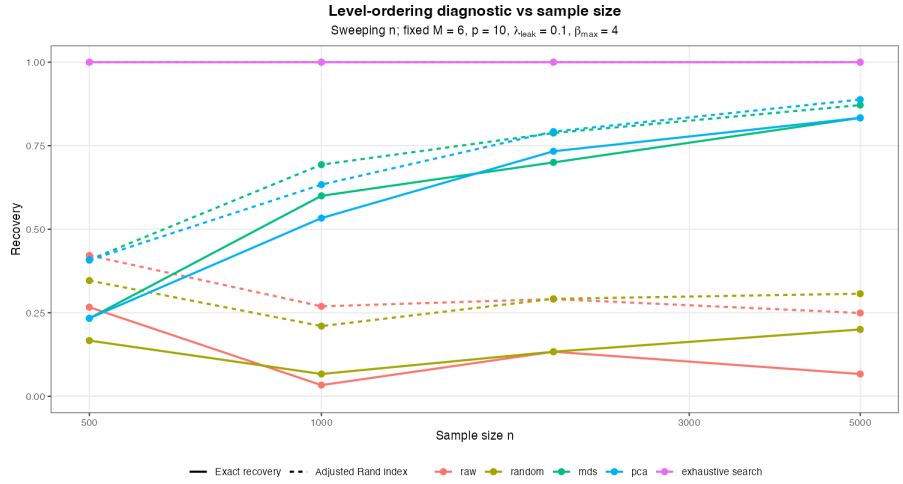

Figure 9: Sample size sweep $n \in \{500, 1000, 2000, 5000\}$ at $M = 6$, $p = 10$, $\lambda_{\text{leak}} = 0.1$, $\beta_{\max} = 4$, binary-slope mechanism.

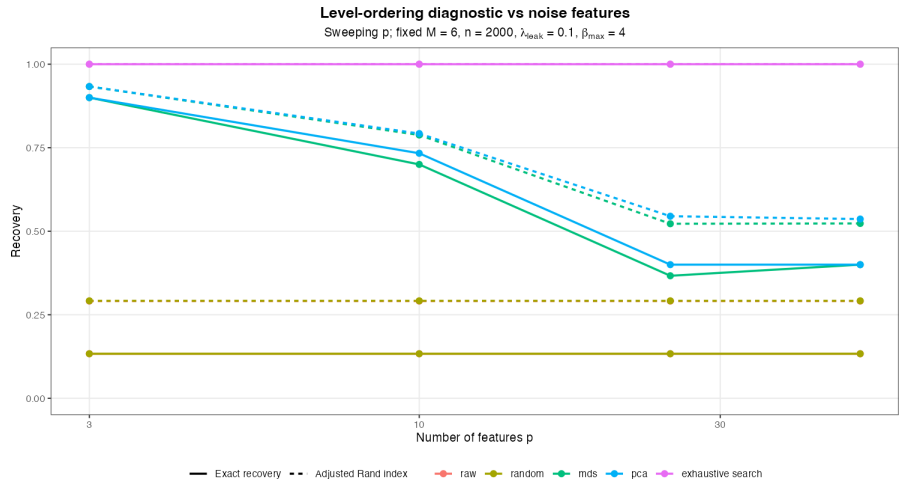

Figure 10: Feature dimension sweep $p \in \{3, 10, 25, 50\}$ at $M = 6$, $n = 2000$, $\lambda_{\text{leak}} = 0.1$, $\beta_{\max} = 4$, binary-slope mechanism.

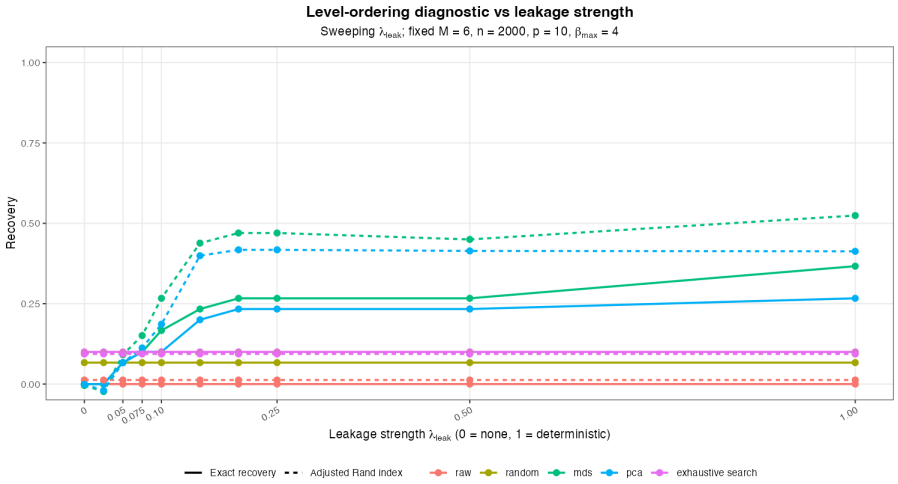

Figure 11: Leakage sweep λ_{leak} under the group mechanism at $M = 6$, $n = 2000$, $p = 10$, $\beta_{\max} = 4$. Each x_3 level has an independent slope $\beta_\ell \sim U[-4, 4]$. Recovery metrics use the $\lceil M/3 \rceil$ levels with the largest $|\beta_\ell|$ as the reference grouping.